Cuaderno de estudio y práctica — Ingeniería de Software

Teoría, práctica y preparación para evaluaciones por asignatura y semana

Diego Saavedra

Índice

1 Cómo usar este cuaderno . . . 9
1.1 Ruta rápida . . . 9
1.2 Un ciclo de práctica . . . 9
1.3 Qué fuentes tienen prioridad . . . 9
1.4 Autoevaluación formativa . . . 10
1.5 Alcance de esta edición . . . 10
Cobertura actual . . . 12
Navegación de la Semana 1 . . . 12
Navegación de la Semana 2 . . . 12
Alcance de las semanas actuales . . . 13
Ciclo de estudio . . . 13

I Aplicaciones Web

2 Clase 1: de una petición a una respuesta . . . 15
2.1 Una página no siempre es un archivo fijo . . . 15
2.2 El ciclo mínimo HTTP . . . 16
2.3 HTML en el cliente; programas en el servidor . . . 16
2.4 PHP y C#: comparación inicial . . . 17
2.4.1 Ejemplo mental emparejado . . . 18
2.5 Una historia breve de PHP . . . 18
2.6 Carpetas: pistas sobre responsabilidades . . . 19
2.7 MVC: una introducción, no una receta completa . . . 19
2.8 Fuentes y trazabilidad . . . 19
2.9 Autoevaluación manual . . . 19

3 Clase 2: valores, tipos y alcance . . . 21
3.1 Un nombre apunta a un valor . . . 21
3.2 Tipos escalares y colecciones iniciales . . . 21
3.3 Comillas e interpolación . . . 22
3.4 Constantes: lo que no debe cambiar . . . 22
3.5 Conversión: predice antes de sumar . . . 22
3.5.1 Ejemplo emparejado: texto y número . . . 22
3.6 Alcance: dónde existe un nombre . . . 23
3.6.1 Variable `static` en PHP . . . 25
3.7 Ejecuta y contrasta las demostraciones . . . 25
3.8 Fuentes y límites . . . 25
3.9 Autoevaluación manual . . . 25

4 Práctica y repaso de la Semana 1 . . . 27
4.1 Punto de partida verificable . . . 27
4.2 Un ciclo para resolver, no para adivinar . . . 27
4.3 Recorrido manual de la entrega existente . . . 27
4.4 Misma intención, dos lecturas . . . 28
4.5 Alcance y constantes: una decisión pequeña . . . 28
4.6 Organización de prácticas web . . . 29
4.7 Autoevaluación manual . . . 29
4.8 Criterios de autoevaluación . . . 29
4.9 Siguiente paso . . . 29

5 Entorno de desarrollo opcional . . . 31
5.1 Qué vas a preparar . . . 31
5.2 Elegir una ruta . . . 31
5.3 Compatibilidad del host . . . 32
5.4 Límite importante de SQL Server . . . 32
5.5 Ruta A: VS Code y terminal Compose . . . 32
5.5.1 Preparación común . . . 32
5.5.2 PHP . . . 33
5.5.3 C# / ASP.NET Core . . . 34
5.5.4 Direcciones y persistencia . . . 34
5.5.5 Si algo falla . . . 35
5.6 Ruta B: Dev Containers . . . 35
5.6.1 Copiar la configuración PHP . . . 35
5.6.2 Copiar la configuración C . . . 35
5.7 Comprobación de aprendizaje, no de plataforma . . . 36
5.8 Referencias de operación . . . 36

6 Semana 2 · Clase 1: tipos, conversiones y Cliente . . . 37
6.1 1. Valores simples y ausencia de valor . . . 37
6.2 2. Convertir no es validar . . . 38
6.2.1 Igualdad: valor y tipo . . . 38
6.3 3. Contratos de tipo . . . 38
6.4 4. Nombres dinámicos bajo control . . . 38
6.5 5. Cliente tipado y operaciones en memoria . . . 39
6.6 Pasos de práctica . . . 39
6.7 Ejercicios y respuestas razonadas . . . 39
6.7.1 Ejercicio 1 . . . 39
6.7.2 Ejercicio 2 . . . 40
6.7.3 Ejercicio 3 . . . 40
6.7.4 Ejercicio 4 . . . 40
6.7.5 Ejercicio 5 . . . 40
6.7.6 Respuestas . . . 40
6.7.7 Respuesta 1 . . . 40
6.7.8 Respuesta 2 . . . 40
6.7.9 Respuesta 3 . . . 40

6.7.10 Respuesta 4 . . . 40
6.7.11 Respuesta 5 . . . 40

7 Semana 2 · Clase 2: formularios, GET y POST . . . 41
7.1 Formularios: contrato de entrada . . . 41
7.2 GET y POST expresan intenciones distintas . . . 41
7.3 Leer, normalizar y validar . . . 42
7.4 Dos destinos POST y respuestas HTTP . . . 42
7.5 Pasos de práctica local . . . 43
7.6 Ejercicios . . . 43
7.6.1 Ejercicio 1: contrato del formulario . . . 43
7.6.2 Ejercicio 2: elegir método . . . 43
7.6.3 Ejercicio 3: múltiples intereses . . . 44
7.6.4 Ejercicio 4: entrada manipulada . . . 44
7.6.5 Ejercicio 5: PRG y alcance . . . 44
7.7 Respuestas razonadas . . . 44
7.7.1 Respuesta 1 . . . 44
7.7.2 Respuesta 2 . . . 44
7.7.3 Respuesta 3 . . . 44
7.7.4 Respuesta 4 . . . 44
7.7.5 Respuesta 5 . . . 44

8 Tablas de consulta HTTP . . . 45
8.1 Métodos: qué le pides al servidor . . . 45
8.2 Seguridad semántica e idempotencia . . . 45
8.3 Dónde van los datos y dónde verlos . . . 46
8.4 Estados: qué respondió el servidor . . . 46
8.5 Fuentes de consulta . . . 47

9 Laboratorio PHP: cinco métodos en DevTools . . . 49
9.1 Ruta rápida . . . 49
9.2 Abrir y preparar Network . . . 49
9.3 GET: parámetros en la URL . . . 49
9.4 POST: formulario . . . 50
9.5 PUT: cuerpo JSON completo de ejemplo . . . 51
9.6 PATCH: cambio parcial . . . 52
9.7 DELETE: simulación explícita . . . 53
9.8 Qué debes poder explicar . . . 54

10 Respuestas de autoevaluación . . . 57
10.1 P1: historia y propósito . . . 57
10.2 P2: HTTP . . . 57
10.3 P3: tipos . . . 57
10.4 P4: interpolación . . . 57
10.5 P5: constantes . . . 57
10.6 P6: alcance . . . 57

10.7 P7: depuración . . . 57
10.8 P8: comprobación . . . 58
10.9 P9: MVC . . . 58
10.10 P10: transferencia . . . 58
10.11 Cómo usar una discrepancia . . . 58
Cobertura actual . . . 60
Alcance de la cobertura actual . . . 60
Ruta de estudio . . . 60

II Diseño y Arquitectura de Software

11 Diseño y arquitectura: de la necesidad a las decisiones . . . 61
11.1 Objetivos de aprendizaje . . . 61
11.2 De entender la necesidad a decidir una forma . . . 61
11.3 Separar para comprender y cambiar . . . 61
11.4 Vistas que responden preguntas distintas . . . 61
11.5 Caso guiado: reservar sin duplicar una promesa . . . 63
11.6 Estilos y cualidades: opciones con consecuencias . . . 63
11.7 Ejercicio breve . . . 63
11.8 Cierre . . . 63

12 Software: tipos, dominios y mitos . . . 65
12.1 Objetivos de aprendizaje . . . 65
12.2 Dos ejes para no mezclar etiquetas . . . 65
12.3 Una herramienta no cambia de categoría por su nombre . . . 65
12.4 Clasificación trabajada . . . 65
12.5 Cuatro mitos que ocultan condiciones . . . 66
12.5.1 «Agregar programadores tarde arregla el atraso» . . . 66
12.5.2 «Si funciona, está terminado» . . . 66
12.5.3 «Cada cambio es fácil» . . . 66
12.5.4 «No hace falta documentación» . . . 66
12.6 Ejercicio breve . . . 66
12.7 Conclusión . . . 66

13 Práctica integrada y autoevaluación de la Semana 1 . . . 69
13.1 Ruta de trabajo . . . 69
13.2 Caso guiado: una reserva sin promesas duplicadas . . . 69
13.2.1 1. Del enunciado a la clasificación . . . 69
13.2.2 2. Decisión de diseño y arquitectura . . . 69
13.2.3 3. Criterio de calidad y comprobación . . . 70
13.2.4 4. Mito que el caso ayuda a evitar . . . 70
13.3 Ejercicio análogo: préstamo de equipos . . . 70
13.3.1 Entregables y criterios observables . . . 71
13.3.2 Lista de revisión sin puntaje . . . 71
13.4 Autoevaluación formativa . . . 71

13.4.1 1. De necesidad a decisión . . . 71
13.4.2 2. Fronteras, acoplamiento y cohesión . . . 71
13.4.3 3. Vista y notación . . . 72
13.4.4 4. Estilo con contexto . . . 72
13.4.5 5. Dos cualidades distintas . . . 72
13.4.6 6. Tipo, dominio y herramientas . . . 72
13.4.7 7. Mito: sumar personas tarde . . . 73
13.4.8 8. Mito: «funciona, está terminado» . . . 73
13.4.9 9. Mito: «cada cambio es fácil» . . . 73
13.4.10 10. Mito: «no hace falta documentación» . . . 73
13.5 Cierre . . . 74

14 Procesos de software: actividades, flujo y trabajo reutilizable . . 75

14.1 Objetivos de aprendizaje . . . 75
14.2 Un proceso organiza el trabajo, no reemplaza el criterio . . . 75
14.3 Flujo: hacer visible cómo avanza el trabajo . . . 75
14.3.1 Del nombre de una fase a una condición observable . . . 77
14.4 Tareas y patrones: distinto nivel de reutilización . . . 77
14.5 Caso guiado: primer flujo para reservar salas . . . 77
14.5.1 1. Aclarar el alcance inicial . . . 77
14.5.2 2. Relacionar actividades y resultados . . . 77
14.5.3 3. Hacer visible el avance . . . 78
14.6 Práctica breve: del trabajo difuso a tareas revisables . . . 78
14.7 Cierre . . . 78

15 Modelos de proceso: comparación, práctica y autoevaluación . . 79

15.1 Ruta de estudio . . . 79
15.2 Cuatro modelos, cuatro énfasis . . . 79
15.3 Cascada: previsibilidad cuando el cambio es limitado . . . 80
15.4 Incremental: entregar capacidades completas por partes . . . 80
15.5 Evolutivo: aprender con versiones y retroalimentación . . . 80
15.6 Espiral: ciclos guiados por riesgo . . . 80
15.7 Caso trabajado: reservas para una red de bibliotecas . . . 80
15.7.1 Evidencia disponible . . . 80
15.7.2 Comparación antes de decidir . . . 81
15.7.3 Decisión razonada . . . 81
15.7.4 Criterios observables . . . 81
15.8 Práctica análoga: agenda de espacios comunitarios . . . 81
15.8.1 Entregable de estudio . . . 81
15.9 Autoevaluación formativa . . . 82
15.9.1 1. Actividad y tarea . . . 82
15.9.2 2. Flujo que vuelve atrás . . . 82
15.9.3 3. Patrón sin receta . . . 82
15.9.4 4. Cascada y estabilidad . . . 83
15.9.5 5. Incremento funcional . . . 83
15.9.6 6. Evolución con control . . . 83
15.9.7 7. Riesgo en espiral . . . 83
15.9.8 8. Dos proyectos, modelos distintos . . . 83
15.9.9 9. Elegir con cuatro evidencias . . . 84

15.9.10 10. Revisar una elección 84
15.10 Alcance de la fuente y cierre 84

16 Gestión Empresarial y Emprendimiento 85
16.1 Cobertura verificada 85
16.2 Ruta de lectura 85

17 Inteligencia Artificial 87
17.1 Cobertura verificada 87
17.2 Ruta de lectura 87

18 Inteligencia de Negocios 89
18.1 Cobertura verificada 89
18.2 Ruta de lectura 89

19 Proyectos Informáticos 91
19.1 Cobertura verificada 91
19.2 Ruta de lectura 91

20 Fuentes y política de uso 93
20.1 Política de fuentes 93
20.2 Bibliografía 93

Bibliografía 95

1. Cómo usar este cuaderno

Este cuaderno de Ingeniería de Software crece por **asignatura y semana** para preparar evaluaciones mediante teoría, ejemplos resueltos, práctica y autoevaluación. Cada parte declara su cobertura actual: no sustituye la documentación oficial de la asignatura ni registra calificaciones.

1.1 Ruta rápida

1. Elige la asignatura y la semana que estás cursando en la navegación.
2. Lee el concepto, predice el resultado de un ejemplo y realiza la práctica.
3. Conserva una observación concreta y contrástala con la retroalimentación.

El libro reúne seis asignaturas en un solo volumen y un solo PDF. Cada enlace abre un índice independiente con su propia cobertura y ruta de lectura:

- Aplicaciones Web
- Diseño y Arquitectura de Software
- Gestión Empresarial y Emprendimiento
- Inteligencia Artificial
- Inteligencia de Negocios
- Proyectos Informáticos

Aplicaciones Web y Diseño y Arquitectura de Software reúnen las Semanas 1 y 2 con capítulos de estudio redactados en esta edición. Los demás índices declaran la evidencia disponible sin mezclar respuestas ni adelantar contenido por inferencia.

1.2 Un ciclo de práctica

Repita este ciclo para cada ejercicio. El objetivo no es acertar a la primera, sino hacer visible qué evidencia sostiene una respuesta.

1. **Prediga.** Antes de ejecutar, escriba qué espera que ocurra y por qué.
2. **Explique.** Describa qué dato entra, qué regla se aplica y qué resultado debería salir.
3. **Ejecute.** Pruebe un ejemplo pequeño en el repositorio de práctica o en su entorno local.
4. **Compruebe.** Compare la salida y los errores con su predicción.
5. **Reflexione.** Registre qué cambió entre la hipótesis y la evidencia, y formule el siguiente experimento.

Los avisos desplegables de pista y solución ayudan a dosificar la ayuda: primero intente resolver; después contraste su razonamiento. En HTML pueden contraerse y en PDF permanecen como bloques claramente etiquetados.

1.3 Qué fuentes tienen prioridad

Prioridad	Uso en este cuaderno
Documentación oficial de la asignatura	Define los temas y la práctica de cada semana. Puede requerir inicio de sesión en Blackboard.
Documentación técnica primaria	Aclara el comportamiento de lenguajes, plataformas y herramientas.
Repositorios de práctica verificados	Ofrecen ejemplos para observar, no requisitos nuevos.
Explicación editorial	Une las fuentes con lenguaje didáctico original; no reemplaza a las anteriores.

Consulte la política de fuentes y bibliografía para las referencias completas y la fecha de consulta. Este texto parafrasea el material del curso; no reproduce exportaciones privadas de Blackboard.

1.4 Autoevaluación formativa

La autoevaluación es manual. Para cada práctica, conserve una predicción, el comando o acción realizada, la evidencia observada y una explicación breve. Revise después las respuestas razonadas, corrija su explicación y decida un paso siguiente. En Arquitectura, las respuestas están junto a la práctica integrada de Semana 1. No hay LMS, puntos, cuentas ni una promesa de dominio automático.

1.5 Alcance de esta edición

La cobertura publicada depende del avance real y las fuentes verificadas de cada asignatura. No se añaden temarios futuros por inferencia. En Aplicaciones Web, Docker y Dev Containers aparecen como rutas de ejecución opcionales: el aprendizaje central es PHP, C# y la web, no una herramienta concreta.

4.8 Criterios de autoevaluación 29
4.9 Siguiente paso 29

5 Entorno de desarrollo opcional 31
5.1 Qué vas a preparar 31
5.2 Elegir una ruta 31
5.3 Compatibilidad del host 32
5.4 Límite importante de SQL Server 32
5.5 Ruta A: VS Code y terminal 33
5.6 Ruta B: Dev Containers 35
5.7 Comprobación de aprendizaje, no de plataforma . 36
5.8 Referencias de operación 36

6 Semana 2 · Clase 1: tipos, conversiones y Cliente 37
6.1 1. Valores simples y ausencia de valor 37
6.2 2. Convertir no es validar 38
6.3 3. Contratos de tipo 38
6.4 4. Nombres dinámicos bajo control 38
6.5 5. Cliente tipado y operaciones en memoria . . . 39
6.6 Pasos de práctica 39
6.7 Ejercicios y respuestas razonadas 39

7 Semana 2 · Clase 2: formularios, GET y POST 41
7.1 Formularios: contrato de entrada 41
7.2 GET y POST expresan intenciones distintas . . . 41
7.3 Leer, normalizar y validar 42
7.4 Dos destinos POST y respuestas HTTP 42
7.5 Pasos de práctica local 43
7.6 Ejercicios 43
7.7 Respuestas razonadas 44

8 Tablas de consulta HTTP 45
8.1 Métodos: qué le pides al servidor 45
8.2 Seguridad semántica e idempotencia 45
8.3 Dónde van los datos y dónde verlos 46
8.4 Estados: qué respondió el servidor 46
8.5 Fuentes de consulta 47

9 Laboratorio PHP: cinco métodos en DevTools 49
9.1 Ruta rápida 49
9.2 Abrir y preparar Network 49
9.3 GET: parámetros en la URL 49
9.4 POST: formulario 50
9.5 PUT: cuerpo JSON completo de ejemplo 51
9.6 PATCH: cambio parcial 52
9.7 DELETE: simulación explícita 53
9.8 Qué debes poder explicar 54

10 Respuestas de autoevaluación 57
10.1 P1: historia y propósito 57
10.2 P2: HTTP 57
10.3 P3: tipos 57
10.4 P4: interpolación 57
10.5 P5: constantes 57
10.6 P6: alcance 57
10.7 P7: depuración 57
10.8 P8: comprobación 58
10.9 P9: MVC 58
10.10 P10: transferencia 58
10.11 Cómo usar una discrepancia 58

I Aplicaciones Web

Esta asignatura reúne las **Semanas 1 y 2**. Combina teoría, ejemplos resueltos, práctica y autoevaluación para preparar la evaluación a partir de las fuentes oficiales y de observaciones propias; no sustituye la documentación del curso ni registra calificaciones.

Cobertura actual

Semana	Tema y propósito	Lectura
1	Petición, respuesta HTTP y responsabilidades web	Fundamentos de la web
1	Variables, tipos, conversiones y alcance en PHP y C#	Valores, tipos y alcance
1	Repaso práctico con predicciones y evidencia	Práctica y repaso
1	Rutas opcionales para ejecutar los ejemplos	Entorno de desarrollo
2 · Clase 1	Tipos, conversiones, variables de variables y Cliente CRUD en PHP/C#	Tipos y Cliente
2 · Clase 2	Formularios GET/POST, validación y PRG en PHP/ASP.NET Core	Formularios, GET y POST
Consulta transversal	Métodos, datos en DevTools/PHP y estados HTTP	Tablas de consulta HTTP
Consulta transversal	Métodos HTTP prácticos observados en DevTools	Laboratorio PHP

Navegación de la Semana 1

Tipo	Recurso
Teoría	Fundamentos de la web y Valores, tipos y alcance
Ejemplos	Demostraciones ejecutables en el capítulo de valores y Laboratorio PHP; rutas opcionales en Entorno de desarrollo
Ejercicios	Práctica y repaso con predicciones; Tablas de consulta HTTP como referencia durante la práctica
Autoevaluación	Autoevaluación de Semana 1 y sus respuestas razonadas
Entregas	Sitio personal y calculadora publicados en aplicaciones-web-semana1

Navegación de la Semana 2

Tipo	Recurso
Teoría y ejemplos · Clase 1	Tipos, conversiones y Cliente
Teoría y ejemplos · Clase 2	Formularios, GET y POST
Ejercicios	Clase 1 y Clase 2

Tipo	Recurso
Respuestas razonadas	Clase 1 y Clase 2
Práctica local	Demos PHP `semana2-tipos.php` / `semana2.php` y MVC `/Semana2/Tipos` / `/Semana2`; instrucciones dentro de cada clase

La Semana 2 se basa en los materiales de ambas clases aportados para esta práctica; no declara una nueva entrega oficial ni una evaluación aprobada. PHP y ASP.NET Core se ejecutan localmente: este libro estático explica y enlaza el recorrido de estudio.

Alcance de las semanas actuales

El recorrido cubre fundamentos de la web, valores y alcance, y la práctica guiada que los conecta. El entorno de desarrollo es opcional: ayuda a observar los ejemplos, pero no reemplaza el razonamiento sobre la petición, el programa y la respuesta. Las respuestas de Semana 1 están en Respuestas de autoevaluación; las de Semana 2 se encuentran al final de cada clase.

Ciclo de estudio

1. **Lee y predice** el resultado antes de ejecutar un ejemplo.
2. **Explica** la entrada, la regla y la salida con tus palabras.
3. **Practica y comprueba** con una observación concreta.
4. **Contrasta** tu razonamiento con la retroalimentación y registra la siguiente duda.

La cobertura se amplía solo cuando la asignatura avance y existan fuentes oficiales verificadas. Esta página delimita las Semanas 1 y 2 disponibles, sin anticipar semanas futuras.

2. Clase 1: de una petición a una respuesta

Meta. Al terminar podrás explicar qué ocurre entre un navegador y una aplicación, y distinguir el lenguaje, la plataforma y las carpetas básicas de un proyecto. No necesitas montar un servidor para leer esta clase: primero sigue el recorrido y luego comprueba tus predicciones con las prácticas de la semana.

Ruta de estudio: predice qué pide el navegador → sigue la respuesta → compara PHP y C# → nombra la responsabilidad de cada carpeta → comprueba tu explicación.

2.1 Una página no siempre es un archivo fijo

Una página **estática** entrega esencialmente el mismo HTML a quienes solicitan la misma URL. Sus archivos pueden vivir en un servidor, pero no requieren ejecutar lógica para fabricar ese contenido. Una página **dinámica** se construye al atender la petición: puede usar parámetros, datos o reglas antes de enviar la respuesta. Dinámico no significa necesariamente complejo ni que deba incluir una base de datos.

Predicción: al escribir una dirección web y pulsar Enter, ¿el navegador recibe PHP o C#? Normalmente recibe una **respuesta HTTP**, cuyo cuerpo puede ser HTML, CSS, JavaScript, una imagen o datos. El código del servidor se ejecuta antes y no se entrega como parte de la página.

Los dos recorridos comparten el mismo ciclo HTTP, pero se muestran por separado para que cada figura permita seguir una sola tecnología.

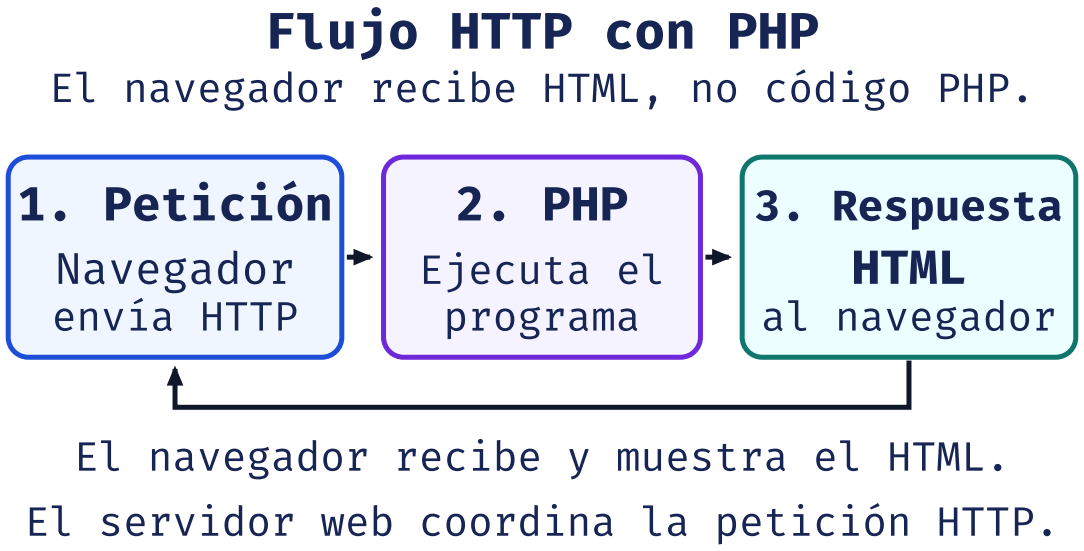


Figura 2.1: Flujo PHP: 1. petición del navegador; 2. PHP ejecuta el programa en el servidor; 3. una respuesta con cuerpo HTML vuelve al navegador.

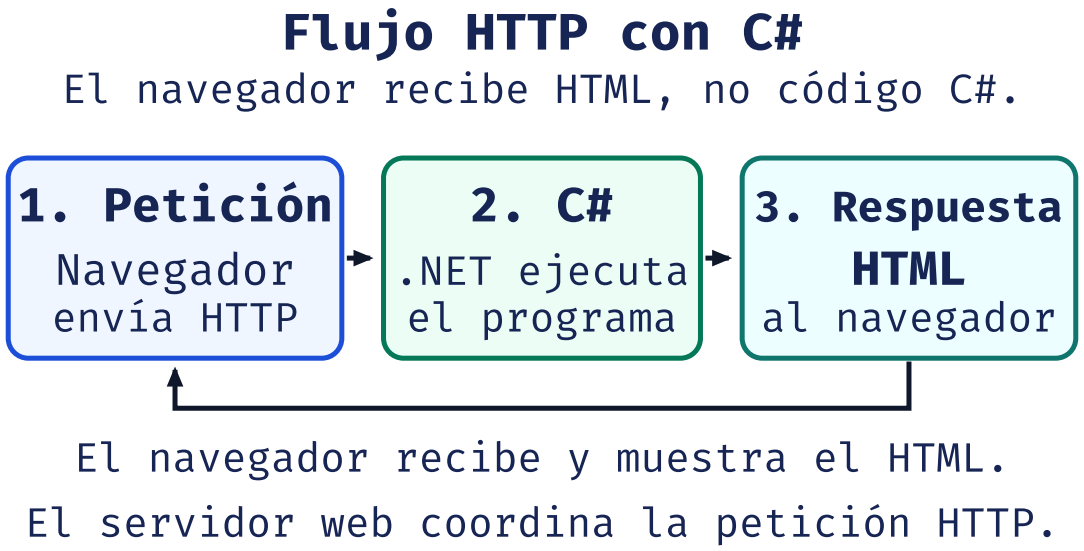


Figura 2.2: Flujo C#: 1. petición del navegador; 2. .NET ejecuta el programa C# en el servidor; 3. una respuesta con cuerpo HTML vuelve al navegador.

2.2 El ciclo mínimo HTTP

1. El **cliente** (por ejemplo, un navegador) envía una petición: método, ruta, cabeceras y, si corresponde, datos.
2. El **servidor** la recibe y el programa decide qué hacer.
3. La respuesta incluye un código de estado, cabeceras y un cuerpo.
4. El navegador interpreta el HTML recibido y solicita recursos adicionales si los encuentra.

Los estados no son mensajes decorativos. `200 OK` indica que la respuesta fue atendida; `404 Not Found` comunica que no se encontró el recurso solicitado; `500 Internal Server Error` señala un fallo no controlado del lado servidor. Un `200` no garantiza que el contenido sea útil, y un `404` no prueba por sí solo que el programa esté roto: expresa el resultado de esta petición.

> **i** Observa antes de concluir
>
> Las herramientas de desarrollo del navegador permiten ver URL, método, estado y cuerpo de una respuesta. Es una comprobación mejor que deducir el resultado solo por lo que parece mostrar la pantalla.

2.3 HTML en el cliente; programas en el servidor

HTML describe la estructura que el navegador renderiza. CSS presenta esa estructura y JavaScript puede modificarla en el cliente. PHP y C# pueden generar HTML o datos en el servidor. Esta división ayuda a diagnosticar: si el navegador muestra una etiqueta literalmente, probablemente llegó como texto; si aparece un error de PHP o .NET, la ejecución ocurrió en el servidor.

No confundas «usar un editor» con «ejecutar una aplicación». Un **editor** escribe y organiza archivos. El **runtime** interpreta o ejecuta el programa. Un **servidor web** recibe HTTP y puede delegar la petición al runtime. En desarrollo pueden estar en el mismo equipo; sus funciones siguen siendo distintas.

Pregunta	Editor	Runtime	Servidor web
¿Escribe `index.php` o `Program.cs`?	Sí	No	No
¿Ejecuta el lenguaje?	No	Sí	Coordina o delega

Pregunta	Editor	Runtime	Servidor web
¿Acepta una petición HTTP?	No	No necesariamente	Sí

2.4 PHP y C#: comparación inicial

PHP es un lenguaje usado con frecuencia para generar respuestas web. En el flujo tradicional, el runtime evalúa el archivo PHP que corresponde a la petición. Un acelerador como OPcache puede conservar código compilado en memoria, por lo que no es correcto afirmar que PHP «siempre recompila todo» en cada solicitud [1].

C# es el **lenguaje** con el que escribimos el programa. .NET es la **plataforma** que ofrece runtime, bibliotecas y herramientas para compilar y ejecutar ese programa. Al compilar, C# produce código intermedio (IL); el runtime aplica compilación JIT cuando necesita ejecutar código. Es una comparación de rutas de ejecución, no una escala de calidad: la elección depende del contexto y del proyecto [2].

Las figuras separan las rutas para que los pasos propios de cada plataforma no se mezclen. La comparación sigue en el texto: ambas producen una respuesta, aunque su preparación y ejecución no son idénticas.

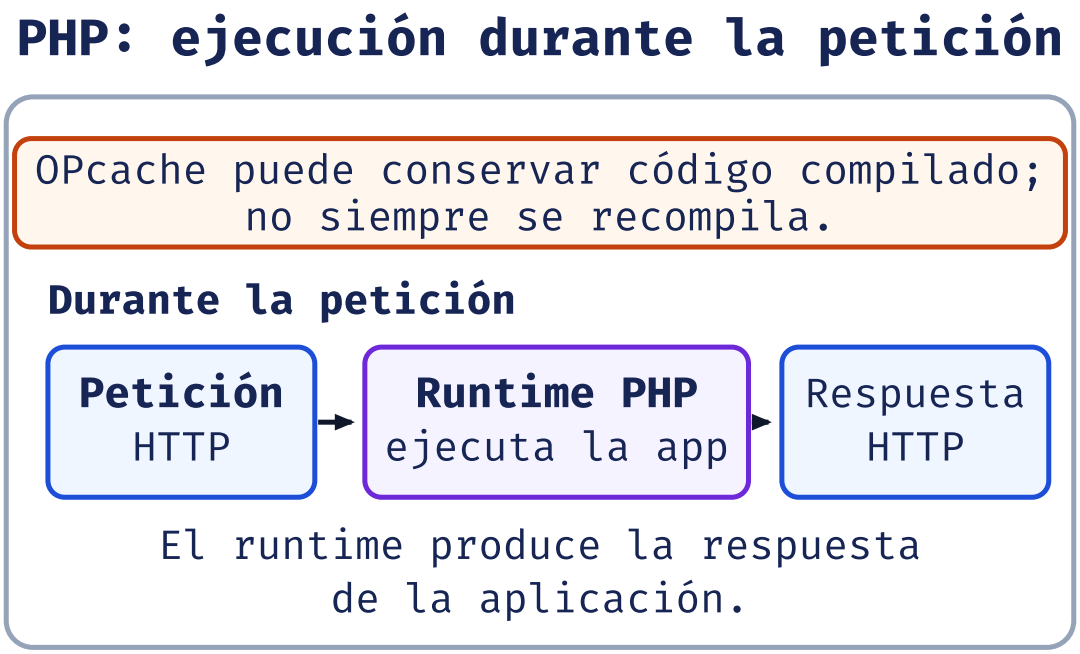


Figura 2.3: Ruta de ejecución de PHP: durante la petición, el runtime ejecuta la aplicación; OPcache puede conservar código compilado.

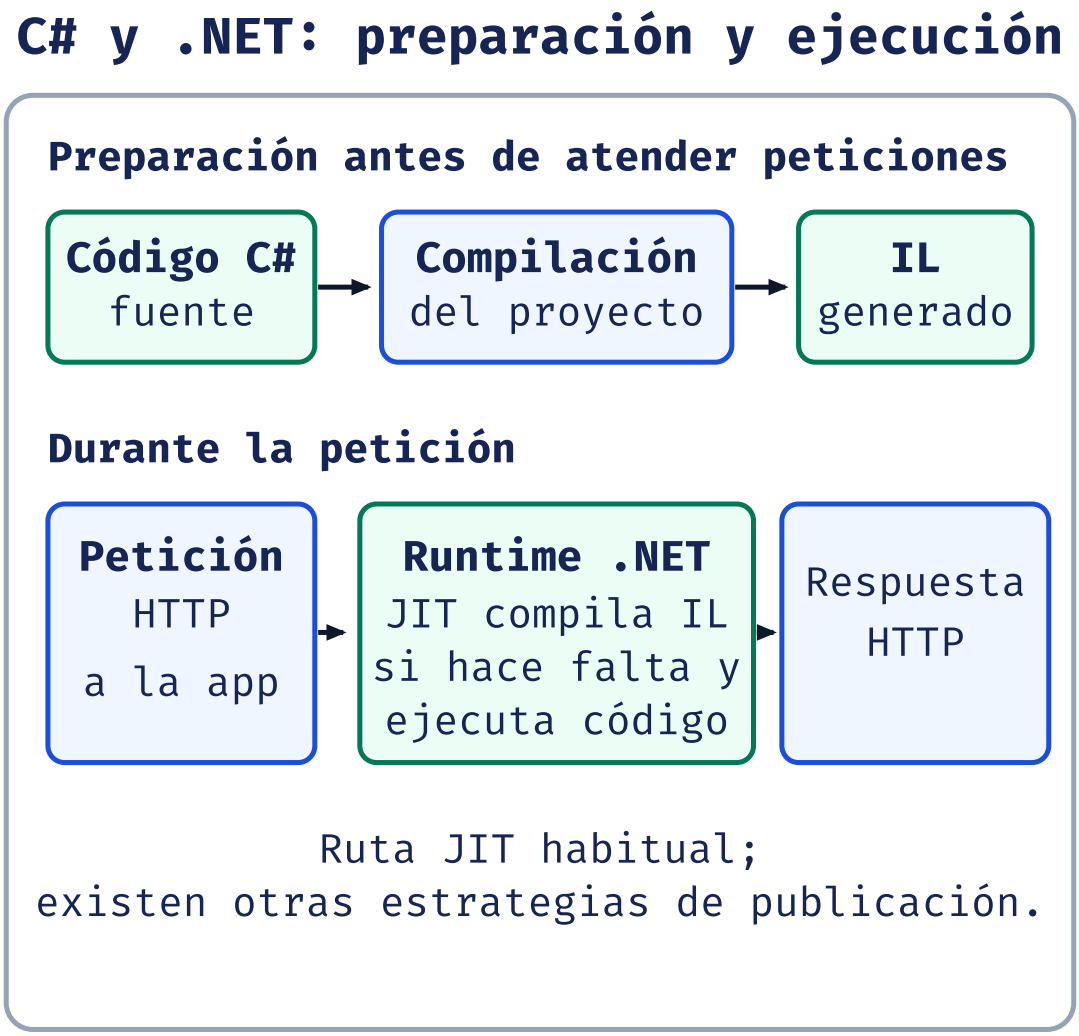


Figura 2.4: Ruta de ejecución de C#: el proyecto se compila a IL antes de atender peticiones; durante la petición, el runtime .NET aplica JIT cuando hace falta y ejecuta el código.

2.4.1 Ejemplo mental emparejado

Supón que una ruta debe saludar a Ada. En PHP el runtime evalúa una instrucción que produce `Hola Ada`; en C#, la aplicación compilada ejecuta una instrucción equivalente y produce el mismo texto. En ambos casos el resultado puede insertarse en una respuesta HTTP. Predice cuál parte ve el navegador: ve el texto final, no la variable ni el paso de compilación.

Por qué importa. Si cambias HTML, estás cambiando lo que el cliente interpreta. Si cambias PHP o C#, estás cambiando cómo el servidor construye la respuesta. Para confirmarlo, inspecciona la respuesta en el navegador y observa que no contiene el código de servidor.

2.5 Una historia breve de PHP

La evolución importa porque explica por qué encontrarás proyectos con estilos diferentes. PHP 5 consolidó gran parte del modelo orientado a objetos; PHP 7 trajo mejoras importantes de rendimiento y tipos; PHP 8.0 incorporó cambios de lenguaje como atributos y unión de tipos. Las versiones 8.1 a 8.3 continuaron refinando el lenguaje, y PHP 8.4 y 8.5 son versiones posteriores documentadas por el proyecto. Esta clase no es un tutorial de sus novedades: al trabajar en un repositorio, verifica la versión real antes de usar una característica [3].

La versión activa es una condición técnica, no un detalle de estilo. Una instrucción válida en una versión reciente puede no funcionar en una instalación anterior. Consulta las notas oficiales de PHP 8.5 y el manual para confirmar disponibilidad, en vez de copiar fragmentos aislados [1], [3].

2.6 Carpetas: pistas sobre responsabilidades

Un punto de partida frecuente separa estas carpetas:

Carpeta	Responsabilidad inicial
`public/`	Punto expuesto al navegador: archivos públicos y entrada web.
`src/`	Código de la aplicación que no debería ser una URL directa.
`views/`	Plantillas o fragmentos destinados a producir la presentación.
`config/`	Configuración de la aplicación, no secretos publicados.

Esta convención no es una ley del lenguaje ni convierte automáticamente un proyecto en MVC. Es una forma de hacer visible qué archivo está expuesto y dónde buscar una responsabilidad. Nunca publiques contraseñas en `public/` ni en un repositorio.

PHP ofrece `include` y `require` para incorporar archivos. Si falta un archivo, `include` emite una advertencia y el script puede continuar; `require` produce un error fatal y detiene la ejecución. Las variantes `include_once` y `require_once` se comportan igual, pero solo cargan cada archivo una vez, lo que evita declarar la misma función dos veces. Para configuración o código indispensable, `require` evita continuar con una aplicación incompleta. Comprueba la ruta y el mensaje; no silencies el problema con operadores de supresión [1].

2.7 MVC: una introducción, no una receta completa

MVC propone separar responsabilidades: un **modelo** representa datos y reglas, una **vista** presenta información y un **controlador** coordina una petición. ASP.NET Core MVC documenta este patrón para aplicaciones web [4]. En Semana 1 basta con reconocer el propósito de esa separación y no colocar una decisión de interfaz en un lugar arbitrario.

El ejemplo de factura PHP existente en el repositorio es monolítico y **no** es una arquitectura MVC de referencia. La aplicación C# ya usa una plantilla MVC, pero eso no convierte toda su lógica de negocio en un catálogo de patrones de dominio. No reestructures esas prácticas durante este repaso: observa, nombra responsabilidades y avanza gradualmente.

> ! Límite de Semana 1
>
> Patrones de diseño, Clean Architecture y TDD requieren práctica deliberada posterior. Aquí no abrimos capítulos nuevos sobre ellos: usamos solo la idea inicial de separar responsabilidades para leer mejor una aplicación MVC.

2.8 Fuentes y trazabilidad

El temario de Clase 1 es la fuente curricular primaria [5]. Para confirmar detalles técnicos consulta el manual oficial de PHP, las notas de PHP 8.5, la referencia de C# y la introducción oficial a ASP.NET Core MVC. Consulta: 2026-10-30.

2.9 Autoevaluación manual

1. ¿Qué diferencia práctica hay entre el cuerpo de una respuesta HTTP y el código PHP?
2. ¿Por qué `404` y `500` describen situaciones distintas?
3. ¿Qué distingue a C# de .NET cuando explicas cómo se ejecuta un programa?

4. ¿Cuándo prefieres `require` a `include` y por qué?

> **i** Retroalimentación razonada
>
> 1. El navegador recibe el cuerpo de la respuesta; PHP se ejecutó antes en el servidor.
> 2. `404` comunica que no se encontró el recurso; `500` indica un error interno al procesar la solicitud. Ambos se verifican en la respuesta, no solo en la página.
> 3. C# es el lenguaje; .NET aporta runtime y bibliotecas que ejecutan IL mediante JIT.
> 4. Cuando el archivo es indispensable, porque detenerse revela una aplicación incompleta en vez de continuar con un estado incierto.

Siguiente paso: en la Clase 2 predice tipos, conversiones y ámbitos antes de confirmarlos con los dos programas de demostración.

3. Clase 2: valores, tipos y alcance

Meta. Declararás valores con intención, anticiparás una conversión y explicarás qué nombre está disponible en cada parte del programa. La meta no es memorizar una lista: es predecir, ejecutar y justificar el resultado en PHP y C#.

> **Ciclo de estudio:** identifica el valor → predice el tipo y el alcance → compara ambos lenguajes → ejecuta el ejemplo → explica cualquier diferencia.

3.1 Un nombre apunta a un valor

Una variable asocia un nombre con un valor que puede cambiar. El nombre debe describir el dato (`$total`, `nombreCliente`) y respetar las reglas del lenguaje. PHP distingue mayúsculas y minúsculas en los nombres de variables: `$total` y `$Total` no son el mismo nombre. C# también distingue `total` de `Total`; por convención suele usar `camelCase` para variables locales y `PascalCase` para constantes y miembros.

En PHP una variable empieza con `$`:

```php
$nombre = 'Ada';
$activo = true;
```

En C# se declara primero el tipo o se permite que `var` lo infiera:

```csharp
string nombre = "Ada";
bool activo = true;
var edad = 20;
```

`var` **no** hace dinámico a C#. El compilador infiere un tipo estático a partir del valor inicial; `edad` sigue siendo `int` y no puede recibir luego una cadena. PHP trabaja habitualmente con tipado dinámico: una variable puede contener valores de tipos diferentes en momentos distintos. Esa flexibilidad no elimina la necesidad de leer conversiones y contratos con cuidado [1], [2].

3.2 Tipos escalares y colecciones iniciales

Empieza por cuatro tipos escalares frecuentes: enteros, números de punto flotante, cadenas y booleanos. PHP también tiene `null`, que expresa ausencia de valor. C# usa `null` para referencias anulables; con tipos de valor, `int?` representa un entero que puede ser nulo. Activa y respeta las advertencias de nulabilidad: no son una garantía absoluta, pero señalan rutas que podrían fallar.

Idea	PHP	C#
Entero	`3`	`int cantidad = 3;`
Decimal	`3.5`	`double precio = 3.5;`
Cadena	`'Ada'` o `"Ada"`	`string nombre = "Ada";`
Booleano	`true`	`bool activo = true;`
Ausencia	`null`	`string? apodo = null;`

Un array de PHP puede actuar como lista o como mapa de claves. En C# conviene nombrar la colección por su intención; para comenzar, un arreglo como `string[] nombres = ["Ada", "Lin"];` almacena elementos del mismo tipo. No conviertas esta introducción en una taxonomía de colecciones: primero reconoce qué datos contiene cada ejemplo.

3.3 Comillas e interpolación

En PHP las comillas simples preservan el texto: `'Hola $nombre'` no reemplaza la variable. Las dobles permiten interpolación: `"Hola {$nombre}"`. Las llaves aclaran el límite del nombre y evitan ambigüedades al rodearlo de texto.

C# usa comillas dobles para `string` y antepone `$` para interpolar:

```
string nombre = "Ada";
string saludo = $"Hola {nombre}";
```

Predice el resultado de los dos fragmentos antes de ejecutarlos. En PHP, una comilla simple muestra `$nombre` literalmente; en C#, sin el prefijo `$`, `{nombre}` también es texto ordinario. La interpolación mejora legibilidad cuando es sencilla; no sustituye validar ni codificar datos según el contexto donde se usarán.

3.4 Constantes: lo que no debe cambiar

Una constante comunica que un valor no cambia durante el programa. En PHP, `const` se declara como parte del código y `define()` crea una constante en tiempo de ejecución. Para una constante conocida y simple, prefiere `const` por claridad:

```
const CURSO = 'PHP-CSharp';
define('ETIQUETA', 'Semana 1');
```

En C#, `const` requiere un valor de compilación. `readonly` es diferente: se aplica a un **campo de un tipo**, no a una variable local. Un campo de instancia `readonly` puede asignarse en su declaración o en el constructor de instancia; un campo `static readonly`, en su declaración o en el constructor estático. Por tanto, `readonly` **no** equivale a una constante de tiempo de compilación.

Este fragmento declara los campos dentro de una clase; no son instrucciones sueltas para pegar dentro de un método:

```
public static class Configuracion
{
    public const string Curso = "PHP-CSharp";
    public static readonly System.DateTime Inicio = System.DateTime.UtcNow;
}
```

El valor de `Inicio` se fija al inicializar el tipo, no al compilar. Desde otro punto del programa puedes leer `Configuracion.Curso` y `Configuracion.Inicio`. Usa `const` cuando el valor pertenece literalmente al código y `readonly` cuando se fija al construir el objeto o inicializar el tipo. La diferencia afecta dónde y cuándo puede asignarse el valor [2].

3.5 Conversión: predice antes de sumar

Una conversión implícita ocurre cuando el lenguaje puede usar un valor como otro tipo sin sintaxis adicional; una explícita declara la intención. En C#, convertir `double` a `int` exige una conversión explícita porque puede perder decimales:

```
double precio = 3.9;
int truncado = (int)precio; // 3
```

En PHP también puedes convertir explícitamente, por ejemplo `(int) '3'`. Antes de hacerlo, pregúntate si el texto realmente representa un número y valida la entrada externa. Convertir no es corregir datos desconocidos.

3.5.1 Ejemplo emparejado: texto y número

Predice. ¿Qué sale al combinar la cadena numérica `'3'` con el número `3`?

```
$resultado = '3' + 3;
// $resultado es 6
```

PHP convierte esta cadena numérica para la operación aritmética y obtiene `6`. En C#, `+` con una cadena realiza concatenación:

```
string resultado = "3" + 3;
// resultado es "33"
```

Por qué. Los operadores se interpretan junto con los tipos de sus operandos. No copies una expresión de un lenguaje al otro suponiendo igual significado. La cadena `'3a'` empieza con un número, pero no es una cadena numérica válida completa: en PHP 8, `'3a' + 3` produce `6` y una advertencia `E_WARNING`. Que se obtenga un resultado no significa que la entrada sea válida; no dependas de esta conversión. Valida y convierte de forma explícita antes de calcular [1].

Comprobación de error. Si esperabas `6` en C#, convierte el texto validado a un número antes de sumar. Si recibes texto externo en PHP, verifica su formato antes de aplicar `+`; no ocultes advertencias para forzar un resultado.

3.6 Alcance: dónde existe un nombre

El alcance responde «¿en qué parte del código puedo usar este nombre?». Una variable local existe dentro de la función o método que la declara. Los parámetros son una forma explícita de entregar valores a esa función; por eso son preferibles a depender de variables globales.

Las reglas se comparan en el texto, mientras cada figura representa un solo lenguaje.

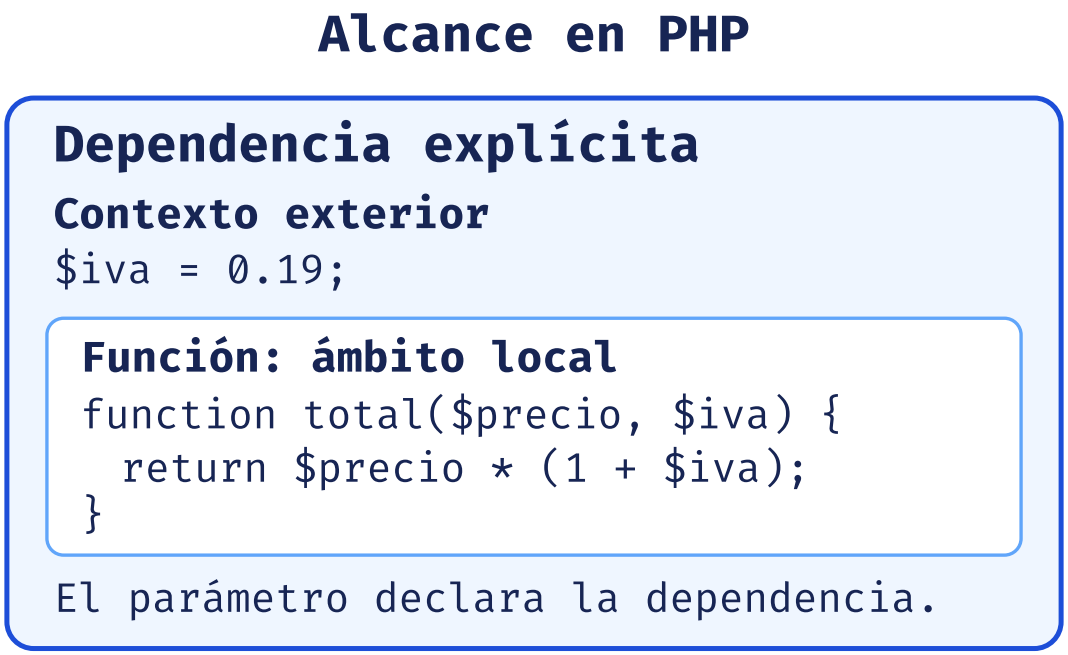


Figura 3.1: Alcance en PHP: una función recibe su dependencia como parámetro explícito dentro de su ámbito local.

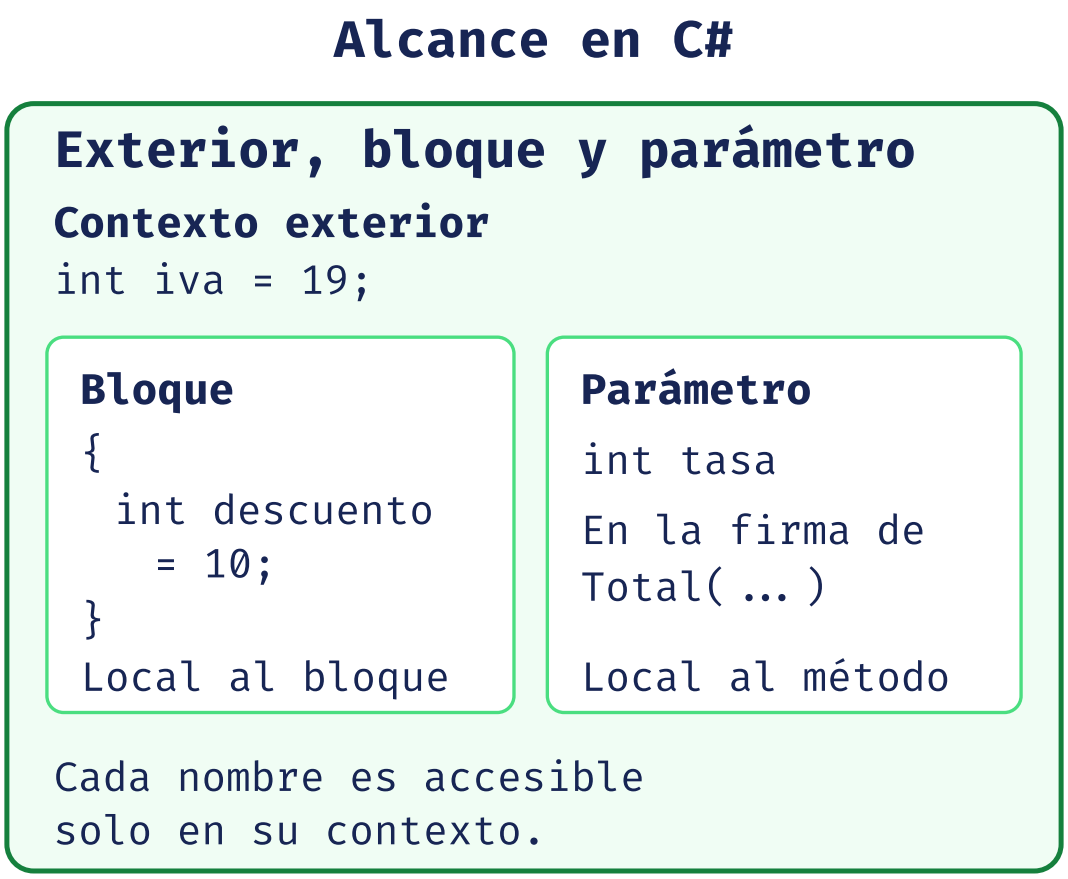


Figura 3.2: Alcance en C#: se distinguen los contextos exterior, de bloque y del parámetro local al método.

Un nombre es visible solo dentro de su ámbito; pasar un parámetro deja la dependencia a la vista.

PHP no crea un nuevo ámbito de variable solo por usar llaves de un `if` o bloque. C# sí delimita variables declaradas dentro de un bloque. Compara estas dos intenciones:

```php
$exterior = 10;
{
    $enBloque = 7;
}
echo $enBloque; // visible: PHP no crea ámbito de bloque
```

```csharp
int exterior = 10;
{
    int enBloque = 7;
}
// Console.WriteLine(enBloque); // error: ya salió del bloque
```

No pruebes el error de C# descomentándolo en la demostración ejecutable: se conserva compilable para que puedas verificar sus resultados. El contraste se explica mejor cuando primero predices y luego reconoces que el compilador evita ese acceso.

En PHP, una función no toma una variable global automáticamente. `global $exterior;` la hace accesible, pero oculta una dependencia. Es más claro recibir `$exterior` como parámetro. En C#, evita el estado global mutable por la misma razón: los parámetros y retornos muestran qué necesita y qué produce una operación.

3.6.1 Variable `static` en PHP

Una variable local `static` conserva su valor entre llamadas a la **misma función durante la petición actual**. No es una memoria compartida y no debe describirse como un contador entre solicitudes HTTP. Cada petición puede iniciar un proceso o contexto distinto; para datos entre peticiones se necesita un mecanismo explícito que no forma parte de esta clase.

C# también puede tener campos `static` asociados al tipo. El ejemplo usa uno para mostrar dos llamadas consecutivas en el mismo proceso de consola, no para prometer persistencia entre reinicios ni entre usuarios web.

3.7 Ejecuta y contrasta las demostraciones

Estos programas pequeños ilustran constantes, alcance y la diferencia de `+`. No son una práctica web MVC ni reemplazan la práctica de HTML/CSS/PHP de Semana 1.

Desde la raíz del repositorio, ejecuta PHP:

```
php ebook/examples/week1/php/variables.php
```

Salida esperada, sin advertencias:

```
CONST|curso=PHP-CSharp|etiqueta=Semana 1
SCOPE|bloque=7|exterior=10|static=1,2
STRING|interpolacion=Hola Ada|suma=6
```

Para C# se usa el proyecto de consola .NET 10. Desde una copia nueva del repositorio, construye primero la imagen pública definida en `csharp/Dockerfile`:

```
docker compose -f csharp/compose.yaml build app
```

Este paso crea `csharp-app:latest` con el nombre de proyecto predeterminado de Compose. Solo construye la imagen: no inicia la aplicación ni SQL Server. Requiere conexión para descargar la imagen base y sus herramientas. Con esa imagen ya disponible, el comando aislado es:

```
docker run --rm --network none --entrypoint sh -v "$PWD/ebook/examples/week1/csharp:/
source:ro" csharp-app:latest -c 'mkdir /work && cp -a /source/. /work/ && cd /work
&& dotnet run --project Week1.csproj --configuration Release --no-launch-profile'
```

Su salida esperada es deliberadamente igual salvo la concatenación:

```
CONST|curso=PHP-CSharp|etiqueta=Semana 1
SCOPE|bloque=7|exterior=10|static=1,2
STRING|interpolacion=Hola Ada|suma=33
```

Las aserciones internas detienen el programa si cambian estos contratos. Si una salida difiere, revisa primero comillas, tipo y ámbito; no atribuyas la diferencia a MVC ni a una petición HTTP porque estos son programas de consola.

3.8 Fuentes y límites

El contenido curricular proviene de Clase 2 [6]. Para detalles de lenguaje consulta el manual oficial de PHP y la referencia oficial de C#. Consulta: 2026-10-30. Temas como patrones, Clean Architecture y TDD se retomarán solo cuando una semana posterior los requiera; aquí la prioridad es dominar valores y alcance observables.

3.9 Autoevaluación manual

1. ¿Por qué `var` no transforma una variable de C# en dinámica?
2. Predice `"Hola $nombre"` y `'Hola $nombre'` en PHP cuando `$nombre` vale `Ada`.
3. ¿Qué resultado producen `'3' + 3` en PHP y `"3" + 3` en C#? Explica la causa.
4. ¿Qué diferencia a `readonly` de `const` en C#?
5. ¿Por qué un parámetro suele ser preferible a leer una variable global?

i Pistas progresivas y retroalimentación razonada

- **Pista 1:** separa «cómo se infiere el tipo» de «cuándo puede cambiar».
- **Pista 2:** mira primero qué comillas habilitan sustitución en PHP.
- **Pista 3:** observa si `+` recibe dos números o una cadena.

`var` infiere un tipo estático en la compilación. Las dobles de PHP producen `Hola Ada`; las simples dejan el nombre literal. PHP suma la cadena numérica y obtiene `6`; C# concatena y obtiene `33`. `readonly` puede asignarse al inicializar o construir, mientras que `const` debe conocerse en compilación. Un parámetro declara la entrada de la operación y reduce dependencias ocultas.

Siguiente paso: usa estas predicciones al revisar la práctica de Semana 1 y anota qué resultado observaste antes de consultar una solución.

4. Práctica y repaso de la Semana 1

Esta práctica no añade una aplicación nueva: organiza lo que ya se estudió para que puedas explicar una decisión antes de escribirla. El objetivo compartido es leer una necesidad pequeña, reconocer sus entradas y resultados, y contrastar cómo PHP y C# expresan la misma idea. PHP y C# no son tareas distintas aquí.

4.1 Punto de partida verificable

La entrega personal existente es el sitio HTML/CSS con una calculadora PHP del commit `ff3d1d7`. No la copies ni la sustituyas: úsala como evidencia previa para observar las cuatro operaciones. El repositorio de práctica actual tiene ejemplos de variables en PHP y C# en el commit `406f4c0`. Son material de comparación, no una afirmación de que ya existe una calculadora C# equivalente.

La consigna de la primera semana pide una página personal HTML/CSS y una calculadora PHP integrada y publicada en control de versiones [7]. Esa es una cita breve del requisito; no se reproduce el material docente.

4.2 Un ciclo para resolver, no para adivinar

Aplica esta secuencia en cada caso. Es útil tanto en el editor como al leer un programa ajeno.

1. **Planifica.** Escribe una frase de propósito: «calcular un resultado a partir de dos números y una operación».
2. **Predice.** Antes de ejecutar, anota qué valor y qué tipo esperas ver.
3. **Separa.** Identifica entrada, procesamiento y salida.
4. **Ejecuta.** Corre solo el ejemplo o la entrega que estás revisando.
5. **Comprueba.** Contrasta el resultado observado con tu predicción.
6. **Explica.** Si no coincide, describe qué dato, operador o conversión puede justificar la diferencia antes de modificar código.

Parte	Pregunta guía	Ejemplo de calculadora
Entrada	¿Qué llega al programa?	dos cantidades y la operación elegida
Procesamiento	¿Qué regla transforma los datos?	`+`, `-`, `*` o `/`
Salida	¿Qué debe poder observarse?	resultado o aviso de caso no calculable

Este esquema no reemplaza el razonamiento sobre HTTP: en una página dinámica, una solicitud llega al servidor y este construye una respuesta. La página estática no necesita calcular con datos de la solicitud; la dinámica sí puede hacerlo [7].

4.3 Recorrido manual de la entrega existente

Abre la entrega personal y conserva una pequeña tabla de observación. No des por hecho que «se ve bien» equivale a que cada operación fue revisada.

Caso declarado	Predicción que anotas	Qué comprobar
`8 + 2`	`10`	que la suma no concatene texto
`8 - 2`	`6`	que se aplique la resta
`8 * 2`	`16`	que se aplique la multiplicación

Caso declarado	Predicción que anotas	Qué comprobar
`8 / 2`	`4`	que el cociente sea el esperado
divisor `0`	no hay división válida	que el caso sea distinguible y explicable
entrada no numérica	no hay cálculo aritmético fiable	qué muestra la entrega y por qué

Los dos últimos casos son **observaciones de la práctica declarada**, no un encargo para construir ahora un marco completo de validación. Si un resultado difiere, registra entrada exacta, salida exacta y el paso de procesamiento que investigarías. Esa nota vale más que ocultar el caso con una corrección rápida.

> Consejo de comparación
>
> Primero formula el algoritmo en lenguaje cotidiano. Después mira cómo cada lenguaje declara sus variables y muestra el resultado. Así evitas confundir la sintaxis con el problema que se desea resolver.

4.4 Misma intención, dos lecturas

En PHP una variable comienza con `$`; en C# su declaración comunica un tipo explícito o permite inferirlo con `var` cuando el valor inicial lo hace claro. Ambos lenguajes permiten componer texto con valores, pero sus reglas concretas no son intercambiables. Consulta las referencias del lenguaje antes de inferir una conversión [1], [2].

```php
<?php
$nombre = "Ana";
$intentos = 2;
echo "$nombre tiene $intentos intentos.";
```

```csharp
string nombre = "Ana";
int intentos = 2;
Console.WriteLine($"{nombre} tiene {intentos} intentos.");
```

Predice primero: ambos fragmentos comunican que Ana tiene 2 intentos. Luego identifica lo que cambia: `$` introduce la variable en PHP, mientras que en C# el `$` pertenece a la cadena interpolada. El ejemplo ilustra variables y salida; no prueba por sí solo una aplicación web completa.

4.5 Alcance y constantes: una decisión pequeña

Una constante representa un valor que no debe reasignarse durante el programa. Una variable representa un dato que puede cambiar según el problema. El alcance indica dónde un nombre es visible. Cuando una función necesita un dato, pasarle un parámetro hace visible esa dependencia en la llamada; no es necesario usar un estado global para este ejercicio.

```php
<?php
const TASA = 0.19;
function precioConImpuesto(float $base): float {
    return $base * (1 + TASA);
}
```

```csharp
const decimal Tasa = 0.19m;
static decimal PrecioConImpuesto(decimal baseImponible)
{
    return baseImponible * (1 + Tasa);
}
```

No memorices estos fragmentos como plantilla universal. Señala en ambos la entrada (`$base`/`baseImponible`), el procesamiento (multiplicar) y la salida (el valor retornado). En PHP, `const` y `define`

tienen reglas de uso distintas; en C#, `const` declara una constante de compilación. Revísalas en la referencia oficial antes de generalizar [1], [2].

4.6 Organización de prácticas web

MVC es el patrón de organización adoptado para las prácticas web: separa la representación, el manejo de solicitudes y los datos del dominio para que sea más fácil localizar responsabilidades [4]. No convierte retroactivamente todo archivo en MVC. En particular, el `php/public/home.php` actual de las facturas es monolítico; este ebook no lo refactoriza y las facturas/persistencia no son requisitos oficiales comprobados de Semana 1 [8].

4.7 Autoevaluación manual

Responde en una libreta antes de abrir las soluciones. Los enlaces estables llevan a respuestas razonadas, no a una calificación ni a una prueba segura.

1. P1: historia y propósito ¿Qué problema ayuda a resolver una página dinámica frente a una estática y qué aspecto de la evolución de PHP revisarías en una fuente oficial?
2. P2: HTTP En una calculadora web, distingue solicitud, procesamiento en servidor y respuesta con tus propias palabras.
3. P3: tipos Predice qué intención expresa `"8" + "2"` en PHP y por qué debes verificarlo en vez de asumir que equivale a la suma C# `8 + 2`.
4. P4: interpolación Predice la salida de los dos ejemplos de Ana y nombra una diferencia sintáctica.
5. P5: constantes ¿Cuándo elegirías una constante en vez de una variable? Da un ejemplo de cada una.
6. P6: alcance Explica por qué un parámetro hace más visible la dependencia de una función que recurrir a `global`.
7. P7: depuración Si `8 / 0` no produce un resultado numérico útil, ¿qué tres observaciones registrarías antes de tocar el código?
8. P8: comprobación Diseña una tabla mínima para verificar las cuatro operaciones, cero y entrada no numérica.
9. P9: MVC ¿Qué afirma y qué no afirma decir que MVC es el patrón adoptado, dado el `home.php` PHP actual?
10. P10: transferencia Para convertir temperaturas, identifica entrada, procesamiento y salida; después escribe una predicción de prueba para PHP y otra para C#.

4.8 Criterios de autoevaluación

Estos son criterios de autoevaluación, no rúbrica docente. Marca cada punto solo si puedes mostrar tu predicción y una observación.

Evidencia personal	Aún no	Sí, con nota concreta
Distinguí entrada, procesamiento y salida en un caso	☐	☐
Revisé las cuatro operaciones y el divisor cero	☐	☐
Registré qué ocurrió con una entrada no numérica	☐	☐
Expliqué una diferencia PHP/C# sin inventar paridad	☐	☐
Justifiqué una constante, un tipo y un parámetro	☐	☐

Una pista o una respuesta te ayuda a comparar tu razonamiento; no demuestra que hayas realizado la práctica. Conserva tus notas y vuelve al ejemplo cuando una predicción falle.

4.9 Siguiente paso

Lee las respuestas razonadas en Respuestas de autoevaluación y, si deseas ejecutar los ejemplos sin cambiar el objetivo de aprendizaje, continúa con Entorno de desarrollo.

5. Entorno de desarrollo opcional

El objetivo sigue siendo aprender PHP y C#, no dominar una herramienta de contenedores. Elige una de las dos rutas de esta página y aplica las mismas comprobaciones de los capítulos **Fundamentos web** y **Variables, tipos y alcance**. Docker Compose y Dev Containers son alternativas de ejecución, no contenidos que debas aprobar para comprender la Semana 1.

! Antes de empezar

En Linux se verificaron los ejemplos de lenguaje y la ruta de ejecución con Docker Compose. No se ejecutaron estas comprobaciones en Windows ni macOS, ni se probó la integración real de VS Code con Dev Containers. Los archivos bajo `ebook/examples/devcontainers/` son ejemplos para **copiar**; no son configuraciones activas y no funcionan ejecutándolos desde esa carpeta.

5.1 Qué vas a preparar

Prepararás **dos escenarios alternativos**, cada uno con dos contenedores: uno para la aplicación y otro para su base de datos. No necesitas instalar PHP, .NET, MariaDB ni SQL Server directamente en tu computadora para esta ruta.

Escenario	Servicio `app`	Servicio `db`
PHP + MariaDB	Apache, PHP y las extensiones `pdo_mysql` y `mysqli`	MariaDB
.NET + SQL Server	Aplicación ASP.NET Core MVC con `dotnet watch`	SQL Server

Una **imagen** contiene el entorno de ejecución; un **contenedor** lo ejecuta. Un **volumen** conserva los archivos de la base de datos aunque detengas el contenedor. **Compose** describe los servicios y crea la red que los conecta. Los archivos `php/compose.yaml`, `csharp/compose.yaml` y sus respectivos `Dockerfile` ya existen: no debes crear otro proyecto para seguir esta práctica.

⚠ Entorno local, no receta de producción

Ambas aplicaciones usan `127.0.0.1:8080`: inicia **una por vez**. Los puertos de las bases también se publican solo en loopback. La práctica .NET utiliza `sa` y `TrustServerCertificate=True`; estas decisiones de desarrollo no deben copiarse como configuración de seguridad de una aplicación en producción.

5.2 Elegir una ruta

Ruta	Úsala cuando	Idea principal
A. VS Code + terminal Compose	quieres editar en tu host y controlar el stack desde una terminal	VS Code es el editor; Compose crea los servicios
B. Dev Containers	ya tienes un contenedor en ejecución o quieres abrir una copia de configuración	VS Code abre el estándar Dev Container; Docker sigue siendo tecnología separada

Dev Containers es una especificación abierta con integración en VS Code; Docker no es un producto de Microsoft [9], [10]. En ambos casos, la aplicación `app` es la que montará el código y el navegador, si se usa, verá la dirección local indicada por Compose.

5.3 Compatibilidad del host

Host	Ruta de Docker documentada	Nota de arquitectura
Windows	WSL2 con Docker Desktop	Intel/AMD y Apple Silicon no aplican igual; consulta los requisitos de Desktop antes de instalar [11]
Linux	Docker Engine; Docker Desktop no es requisito	la verificación actual fue aquí
macOS	Docker Desktop es el flujo documentado; Colima es una alternativa de compatibilidad no validada aquí	distingue Intel/AMD de Apple Silicon antes de elegir la práctica C# con base de datos

No ejecutes `sudo`, cambios de grupo, instalaciones globales ni cambios de seguridad del host desde esta guía. Sigue la documentación oficial de tu plataforma para preparar Docker [11]. Que un host pueda ejecutar un contenedor simple no prueba que toda la práctica, y especialmente la base de datos, sea compatible.

5.4 Límite importante de SQL Server

La práctica C# usa SQL Server como servicio `db`. La documentación primaria de SQL Server en contenedores Linux indica soporte para Linux x86-64 (Intel/AMD). La emulación QEMU o Rosetta no está probada ni soportada [12]. Por ello no se promete que el stack C# con SQL Server funcione de manera nativa en Apple Silicon, ni se reemplaza silenciosamente por otra base de datos.

Los ejemplos de lenguaje que no dependen de la base de datos pueden ejecutarse con imágenes multi-arquitectura de PHP o del SDK .NET cuando estén disponibles. Otra opción es un SQL Server x64 remoto, pero exige configurar explícitamente la conexión: no viene configurado por este repositorio. Esta limitación no cambia las comparaciones de variables, tipos, constantes o alcance.

5.5 Ruta A: VS Code y terminal Compose

Esta ruta no requiere una ventana especial de VS Code. Ejecuta los comandos siguientes desde la **raíz del repositorio**, no desde `php/public` ni `csharp/src`.

5.5.1 Preparación común

1. Abre el proyecto. Si todavía no tienes una copia, puedes obtenerla así:

```
git clone https://github.com/statick88/aplicaciones-web-php-csharp.git
cd aplicaciones-web-php-csharp
```

Si ya tienes una copia, abre su carpeta: no vuelvas a clonarla. En la raíz deben estar las carpetas `php/`, `csharp/` y `ebook/`.

2. Comprueba Docker y Compose.

```
docker --version
docker compose version
docker info
```

Los dos primeros comandos identifican las herramientas; `docker info` comprueba el acceso al motor. Si este último falla, resuelve la preparación de Docker con la documentación de tu plataforma antes de investigar PHP o .NET. No necesitas instalar los lenguajes ni los motores de base de datos en el host. La primera construcción sí necesita acceso a los registros para descargar las imágenes.

3. Decide qué escenario usar. Los dos necesitan el puerto `8080` del host; MariaDB publica además `3306` y SQL Server `1433`. Si otra aplicación ocupa uno de esos puertos, identifica el conflicto antes de arrancar; no borres datos ni modifiques servicios del host automáticamente.

> ⚠ Configura las contraseñas antes del primer arranque
>
> Los Compose incluyen contraseñas de respaldo solo para desarrollo. Los archivos `.env` que prepararás están ignorados por Git: no publiques sus valores ni los incluyas en capturas. Los marcadores entre `<` y `>` deben reemplazarse completos por contraseñas propias antes de ejecutar los comandos.
>
> Si ya habías iniciado una base y su volumen contiene datos, **cambiar `.env` no equivale a cambiar la contraseña almacenada en esa base**. Verifica las credenciales existentes o realiza un cambio explícito dentro del motor; no elimines el volumen para corregir una discrepancia.

5.5.2 PHP

El escenario usa `php/Dockerfile` y `php/compose.yaml`. El primero prepara Apache, PHP 8.5.12 y las extensiones `pdo_mysql` y `mysqli`; Compose conecta `app` con MariaDB. Puedes consultar también la guía PHP del repositorio.

1. Crea `php/.env` con tu editor. Usa dos contraseñas distintas:

```
MARIADB_PASSWORD='<CLAVE_PROPIA_PARA_LA_APLICACION>'
MARIADB_ROOT_PASSWORD='<OTRA_CLAVE_PROPIA_PARA_ROOT>'
```

`MARIADB_PASSWORD` configura tanto el usuario de la base como `DB_PASSWORD` en la aplicación. La aplicación usa el usuario `app` y la base `facturas`, no el usuario administrador `root`. No tienes que crear esos nombres manualmente: MariaDB los prepara mediante las variables del Compose en su inicialización.

2. Valida la configuración y levanta el escenario. Si habías iniciado el escenario C#, detenlo primero como se indica en su sección de apagado.

```
docker compose --env-file php/.env -f php/compose.yaml config --quiet
docker compose --env-file php/.env -f php/compose.yaml up -d --build
```

`--env-file` selecciona explícitamente las variables, `-f` el archivo del escenario, `--build` construye la imagen de la aplicación y `-d` deja los servicios ejecutándose en segundo plano. El primer comando valida el modelo sin imprimir las contraseñas; no comprueba que la base acepte conexiones.

3. Revisa el estado.

```
docker compose --env-file php/.env -f php/compose.yaml ps
```

Busca los servicios `app` y `db`. La base puede aparecer inicialmente como `starting`; `app` depende de que la comprobación de salud de MariaDB termine correctamente. Si hay un problema, consulta cada servicio por separado:

```
docker compose --env-file php/.env -f php/compose.yaml logs --tail=80 db
docker compose --env-file php/.env -f php/compose.yaml logs --tail=80 app
```

4. Comprueba la aplicación y un guardado real. Abre `http://127.0.0.1:8080`. Ver una página confirma que responde PHP/Apache, pero no prueba por sí solo que PHP pueda guardar en MariaDB.

Para comprobar el recorrido completo, abre `http://127.0.0.1:8080/home.php`, completa el formulario con **datos ficticios** y presiona **Guardar**. En este proyecto, `php/public/home.php` abre la conexión mediante PDO al recibir el POST, crea la tabla `facturas` con `CREATE TABLE IF NOT EXISTS` y después ejecuta el `INSERT` parametrizado. No hay un archivo SQL de inicio que debas importar. El resultado esperado incluye **«Guardada en MariaDB»** y el identificador de la fila. Si aparece **«No se pudo guardar en la base de datos»**, revisa el error y las credenciales; no consideres exitoso el ejercicio solo porque el formulario se haya mostrado.

5. Edita el código y detén sin borrar datos. El proyecto PHP completo está montado en `/var/www/html`, pero Apache solo sirve `public/` como DocumentRoot: la lógica (`src/`), las plantillas (`views/`) y la configuración (`config/`) quedan fuera de la URL. Para cambios de PHP normalmente basta con guardar el archivo y recargar el navegador. Reconstruye si modificas el `Dockerfile` o sus extensiones.

```
docker compose --env-file php/.env -f php/compose.yaml stop
```

Para volver a iniciar, reutiliza el mismo comando `up -d --build` del paso 2.

5.5.3 C# / ASP.NET Core

El escenario usa `csharp/Dockerfile` y `csharp/compose.yaml`: un contenedor con SDK .NET 10.0.403 y `dotnet watch`, y otro con SQL Server 2022. Consulta también la guía C# del repositorio. Antes de empezar, respeta el límite x86-64 explicado arriba. El Compose declara `ACCEPT_EULA=Y`: revisa los términos de Microsoft antes de utilizar la imagen.

1. Libera el puerto y crea `csharp/.env`. Si ya preparaste PHP con los pasos anteriores y sigue encendido, detenlo:

```
docker compose --env-file php/.env -f php/compose.yaml stop
```

Si comienzas directamente por .NET y no has preparado PHP, omite ese comando. Luego crea el archivo con tu editor:

```
MSSQL_SA_PASSWORD='<CLAVE_PROPIA_FUERTE_PARA_SQL_SERVER>'
```

Reemplaza el marcador por una contraseña propia; se recomienda al menos 16 caracteres combinando mayúsculas, minúsculas, números y símbolos como `!`, `_` o `-`. Evita `;` en este ejemplo: la contraseña se interpola directamente en la cadena de conexión. El mismo valor configura SQL Server y la variable `ConnectionStrings__Facturas` de la aplicación.

2. Valida y construye.

```
docker compose --env-file csharp/.env -f csharp/compose.yaml config --quiet
docker compose --env-file csharp/.env -f csharp/compose.yaml up -d --build
```

No necesitas ejecutar `dotnet new` ni instalar el SDK en el host: el proyecto MVC ya está en `csharp/src` y el SDK está dentro de `app`.

3. Espera a SQL Server y revisa los registros.

```
docker compose --env-file csharp/.env -f csharp/compose.yaml ps
docker compose --env-file csharp/.env -f csharp/compose.yaml logs --tail=100 db
docker compose --env-file csharp/.env -f csharp/compose.yaml logs --tail=100 app
```

La comprobación de salud de `db` ejecuta una consulta con `sqlcmd`; `app` espera a que termine correctamente. Ese estado confirma que responde el motor, no que el formulario de la aplicación ya haya guardado una factura.

4. Comprueba las migraciones y el formulario. Al iniciar, `Program.cs` ejecuta `Database.Migrate()`: aplica las migraciones pendientes de EF Core en la base `Facturas`. Para este arranque no debes ejecutar manualmente `dotnet ef database update`. Si una migración falla, la aplicación puede no iniciar: revisa los registros de `app`, no solo los de SQL Server.

Abre `http://127.0.0.1:8080` y completa el formulario con datos ficticios. El POST llega a `HomeController`, agrega la entidad y llama a `SaveChanges()`. El envío exitoso vuelve mostrando los datos; esta aplicación no tiene un mensaje específico de «conexión correcta» ni un endpoint de salud propio. No inventes esa evidencia: el recorrido que debes comprobar es formulario, POST y guardado.

5. Edita y detén el escenario. `csharp/src` está montado en `/app`; `dotnet watch` detecta cambios y recompila durante el desarrollo.

```
docker compose --env-file csharp/.env -f csharp/compose.yaml stop
```

Para retomar, usa el mismo comando `up -d --build` del paso 2.

5.5.4 Direcciones y persistencia

Servicio	Desde el navegador o una herramienta del host	Desde `app`
Aplicación PHP o .NET	`http://127.0.0.1:8080`	No aplica
MariaDB	`127.0.0.1:3306`	`db:3306`
SQL Server	`127.0.0.1:1433`	`db:1433`

Dentro de `app`, `localhost` es el propio contenedor de la aplicación. La base se encuentra por el nombre de servicio **`db`**, que resuelve la red de Compose. No hay phpMyAdmin ni un administrador web de SQL Server configurados: la página en `8080` es la aplicación de práctica.

MariaDB guarda sus archivos en un volumen montado en `/var/lib/mysql`; SQL Server en `/var/opt/mssql`. Son datos distintos del código montado desde `src`. `stop` conserva contenedores y volúmenes. Reutiliza el mismo escenario y nombre de proyecto: cambiar de proyecto puede seleccionar otros volúmenes. **No ejecutes `down -v` como solución de problemas:** puede eliminar los datos.

5.5.5 Si algo falla

Síntoma	Qué revisar primero
Docker no responde o no permite el acceso	`docker info` y la preparación del motor en tu host
No encuentra `.env` o `compose.yaml`	La raíz del repositorio y los argumentos `--env-file` y `-f`
Puerto ocupado	El otro escenario o un servicio local que ya usa `8080`, `3306` o `1433`
`db` permanece iniciando o queda `unhealthy`	Registros de `db`, recursos disponibles y, para SQL Server, contraseña y arquitectura
Error de autenticación	Coincidencia de credenciales y si el volumen ya tenía otra contraseña; no lo borres
PHP muestra el formulario pero no guarda	POST a `/home.php`, mensaje de error y conexión PDO
.NET no inicia o falla al enviar	Registros de `app`, migraciones y conexión usada por `SaveChanges()`

Que un contenedor figure en `ps` no prueba todo el escenario. Separa siempre cuatro comprobaciones: configuración válida, servicios disponibles, respuesta HTTP y persistencia de un registro de prueba. No se promete aquí ejecución en hosts o integraciones que no se hayan verificado.

5.6 Ruta B: Dev Containers

Un Dev Container abre el proyecto dentro de un contenedor para que el editor y la terminal integrada compartan ese contexto. No reemplaza Compose: esta configuración le indica a VS Code qué archivo Compose y qué servicio usar. Puedes adjuntarte a un contenedor `app` ya iniciado o copiar una configuración al proyecto y después abrir la carpeta en un contenedor.

5.6.1 Copiar la configuración PHP

1. Desde la **raíz del repositorio**, crea manualmente `php/.devcontainer/` si aún no existe y copia el contenido de `ebook/examples/devcontainers/php.json` a `php/.devcontainer/devcontainer.json`.
2. Confirma que `dockerComposeFile` queda como `../compose.yaml`, que el servicio es `app` y que `workspaceFolder` es `/var/www/html`.
3. En VS Code usa el flujo de Dev Containers para abrir la carpeta `php/` o adjúntate al `app` PHP que ya esté ejecutándose.
4. Ejecuta la comprobación de variables indicada en la guía PHP y registra el resultado observado. No se afirma aquí que una extensión del editor esté instalada.

5.6.2 Copiar la configuración C

1. Desde la **raíz del repositorio**, crea manualmente `csharp/.devcontainer/` si aún no existe y copia el contenido de `ebook/examples/devcontainers/csharp.json` a `csharp/.devcontainer/devcontainer.json`.
2. Confirma `dockerComposeFile: ../compose.yaml`, servicio `app` y `workspaceFolder: /app`.
3. Abre `csharp/` mediante el flujo Dev Containers o adjúntate al `app` C# que ya esté ejecutándose, respetando antes el límite x86-64 de SQL Server.

4. Usa la comprobación descrita en la guía C# y explica qué verificó; no la confundas con una prueba de todas las rutas web.

Los ejemplos deliberadamente no montan el socket Docker, no usan modo `privileged` y no fuerzan apagado automático: `shutdownAction` es `none`. No agregan redes, restauraciones de paquetes ni comandos automáticos. Eso deja las decisiones de host y de ciclo de vida bajo tu control.

5.7 Comprobación de aprendizaje, no de plataforma

Después de cualquiera de las rutas, vuelve al objetivo de Semana 1:

1. elige un valor de entrada y predice una salida;
2. localiza entrada, procesamiento y salida en el ejemplo;
3. contrasta PHP y C# sin exigir que tengan la misma pantalla;
4. anota si el límite fue del código, del contenedor, del puerto o de la compatibilidad de SQL Server.

Si no puedes iniciar el stack C# en Apple Silicon, aún puedes estudiar los fragmentos y completar la comparación conceptual. No presentes esa lectura como ejecución verificada del servicio SQL Server.

5.8 Referencias de operación

- Documentación de Docker Engine para la preparación específica del host [11].
- Especificación Dev Container y uso de contenedores en VS Code para el flujo del editor [9], [10].
- SQL Server en contenedores Linux para el límite x86-64 y soporte de emulación [12].

6. Semana 2 · Clase 1: tipos, conversiones y Cliente

Esta clase conecta valores simples con decisiones observables en código: qué tipo llega, qué conversión se hace y qué ocurre al gestionar un cliente. Los ejemplos comparan PHP y C# sin suponer que sus reglas son idénticas. Para referencias de lenguaje, consulta el manual de PHP y la referencia de C# [1], [2].

El alcance sigue el material de clase aportado como `semana2.md`, contrastado con las prácticas locales de PHP y C#; no implica acceso nuevo a Blackboard [13].

6.1 1. Valores simples y ausencia de valor

Un escalar contiene un valor: `int` (entero), `float`/`double` (decimal), `string` (texto) o `bool` (verdadero/falso). `null` no es un quinto escalar: representa la ausencia de valor. PHP infiere estos tipos desde el valor; C# los declara o los infiere con `var` y distingue una referencia opcional con `string?`.

```php
$cantidad = 3;        // int
$precio = 12.50;      // float
$producto = "Teclado USB"; // string
$enStock = true;      // bool
$descuento = null;    // ausencia de valor
$subtotal = $cantidad * $precio;
```

Esperado: `subtotal` vale `37.5` y es decimal porque interviene `float`. **Observado en el ejemplo:** las comprobaciones `is_int`, `is_float`, `is_string`, `is_bool` e `is_null` corresponden a esos cinco valores. Para mostrar dinero se puede formatear o redondear; no se debe asumir precisión binaria exacta para operaciones como `0.1 + 0.2`.

En la demostración C#, `int cantidad`, `double precio`, `string producto`, `bool enStock` y `string? descuento` expresan la misma intención. El cálculo produce `37.5`; el tipo decimal de punto flotante tampoco convierte una suma en una regla de validación.

```csharp
using System.Globalization;

var datos = new Dictionary<string, string> { ["ciudad"] = "Ambato" };
double precio = double.Parse("8.5", CultureInfo.InvariantCulture);
bool numerosIguales = int.Parse("1") == int.Parse("01");
bool textosIguales = string.Equals("1", "01", StringComparison.Ordinal);

Console.WriteLine($"{precio};          {numerosIguales};          {textosIguales};
{datos["ciudad"]}");
Console.WriteLine(Busqueda.BuscarCliente(15));
Console.WriteLine(Busqueda.BuscarCliente("0102"));

static class Busqueda
{
    public static string BuscarCliente(int id) => $"BuscarPorId({id})";
                public    static    string    BuscarCliente(string    cedula)    =>
$"BuscarPorCedula(\"{cedula}\")";
}
```

Esperado: `precio` es `8.5`, `numerosIguales` es `true`, `textosIguales` es `false` y el diccionario entrega `Ambato` para `"ciudad"`. Las dos sobrecargas conservan separado buscar por id de buscar por cédula.

6.2 2. Convertir no es validar

PHP puede convertir implícitamente cuando una operación lo requiere y permite casts explícitos. Un cast intenta interpretar un valor; no confirma que el texto completo tenga el formato permitido.

```
var_dump("10" + 5);          // int(15)
var_dump((int) "125abc");    // int(125)
var_dump((float) "8,5");     // float(8)
var_dump((bool) "0");        // bool(false)
```

Esperado: el cast conserva el prefijo numérico de `"125abc"`, la coma no es separador decimal para ese cast y `"0"` se vuelve `false`. **Observado:** esos son precisamente los valores comprobados en `php/tests/semana2-tipos.php`. Por eso aceptar `125abc` o `8,5` como entrada numérica válida sería un error de validación, aunque el cast devuelva algo. En C#, la demostración analiza `"8.5"` con `double.Parse` y cultura invariante; para una entrada externa se decide y verifica el formato antes de usar el número.

También son falsos al convertir a booleano `0`, `""`, `[]` y `null`. No los uses como sustituto de una regla de negocio: una regla debe decir qué ausencia, texto o número acepta.

6.2.1 Igualdad: valor y tipo

En PHP 8, `0 == "a"` es `false`; `"1" == "01"` es `true` por comparación numérica, mientras `"1" === "01"` es `false` porque exige tipo y valor. Prefiere $\equiv$ cuando comparas datos que ya tienen tipo conocido. C# no hace ese *type juggling*: el ejemplo convierte ambos textos a entero para comparar sus números y usa `string.Equals( ... , StringComparison.Ordinal)` para contrastar los textos.

6.3 3. Contratos de tipo

`declare(strict_types=1);` cambia cómo PHP trata llamadas hechas desde ese archivo: una cadena como `"3"` no se acepta silenciosamente por un parámetro `int`. No impide que un `int` se use donde el parámetro declara `float`.

```
<?php
declare(strict_types=1);

function calcularTotal(int $cantidad, float $precio,
    ?float $descuento = null): float {
    $total = $cantidad * $precio;
    if ($descuento !== null) {
        $total -= $total * $descuento;
    }
    return round($total, 2);
}
```

Esperado: `calcularTotal(3, 12.5)` resulta `37.5` y, con `0.10`, `33.75`. **Observado:** ambas llamadas pasan en la prueba; `calcularTotal("3", 12.5)` lanza `TypeError` desde la llamada estricta. `?float` permite un `float` o `null`. PHP además declara `int|string` para una clave de búsqueda; C# no tiene ese parámetro unión: dos sobrecargas, una para `int` y otra para `string`, conservan la intención de cada entrada.

6.4 4. Nombres dinámicos bajo control

PHP permite que una variable determine el nombre de otra. Es útil para entender el lenguaje, pero no debe construirse desde datos de usuario.

```
$campo = "ciudad";
$$campo = "Ambato";
$suma = 0;
for ($i = 1; $i <= 3; $i++) {
    ${"nota$i"} = $i * 3;
    $suma += ${"nota$i"};
}
$datos = [$campo => "Ambato"];
```

Esperado: existe `$ciudad` con `Ambato` y la suma de las notas es `18`. **Observado:** el ejemplo comprueba ambos resultados. El arreglo asociativo `$datos['ciudad']` logra la misma consulta de manera visible y recorrible. C# no tiene `$$campo`; usa `Dictionary<string, string>` para una clave dinámica controlada, también con `ciudad = Ambato`.

6.5 5. Cliente tipado y operaciones en memoria

`Cliente` reúne un `id` entero, nombre y correo de texto, `activo` booleano y teléfono opcional. El repositorio usa el id como clave de un arreglo PHP o un `Dictionary<int, Cliente>` C#. Sus operaciones son:

Operación	Efecto observado	Id inexistente
registrar	guarda por id; registrar el mismo id **reemplaza** el cliente anterior	no aplica
consultar	devuelve el cliente o `null`	`null`
listar	devuelve los clientes guardados	lista vacía si no había datos
actualizarCorreo	cambia el correo y devuelve `true`	no cambia datos; devuelve `false`
eliminar	quita el cliente y devuelve `true`	no cambia datos; devuelve `false`

El siguiente es un extracto: presupone que ya se cargaron las clases `Cliente` y `ClienteRepositorioMemoria` de `php/src/semana2-cliente.php`.

```
$repo = new ClienteRepositorioMemoria();
$repo→registrar(new Cliente(1, "Ana Pérez", "ana@correo.com"));
$repo→actualizarCorreo(1, "ana.perez@correo.com");
$noExiste = $repo→consultar(999); // null
```

Esperado: Ana queda con el correo actualizado y `999` no crea ni modifica un cliente. **Observado:** las pruebas cubren consultar, listar, actualizar y eliminar, incluidos los id `999`; no hay una regla de rechazo de duplicados: el registro por la misma clave reemplaza el valor según la fuente.

El repositorio de las demostraciones existe solo en memoria. En una ejecución PHP termina al acabar la solicitud; crear otro repositorio no conserva sus clientes. La acción `Tipos` de C# también crea uno nuevo por solicitud. La aplicación de formularios C# usa una lista serializada en sesión para otro recorrido, por lo que puede reaparecer durante esa sesión; eso no convierte al repositorio de la demostración en una base de datos.

6.6 Pasos de práctica

1. Predice tipo y valor de cada cast antes de ejecutarlo; separa conversión de aceptación de entrada.
2. Ejecuta desde la raíz `php php/tests/semana2-tipos.php` y registra la salida.
3. Ejecuta desde la raíz `php php/tests/semana2-cliente.php`; cambia solo una predicción en tus notas, no las reglas del repositorio.
4. Compara el resultado con `Semana2/Tipos` en C#: identifica qué hace el cast/parsing y qué dato queda solo en memoria.
5. Continúa con formularios, GET y POST para ver cómo un texto llega por una solicitud. La navegación de la asignatura está en Aplicaciones Web.

6.7 Ejercicios y respuestas razonadas

6.7.1 Ejercicio 1

¿Por qué `(int) "125abc"` no valida un id?

6.7.2 Ejercicio 2

Predice `calcularTotal(3, 12.5, 0.10)` y explica qué ocurriría con `"3"` como primer argumento en la llamada estricta.

6.7.3 Ejercicio 3

Compara `"1" == "01"` con `"1" === "01"`. ¿Qué información exige la segunda comparación?

6.7.4 Ejercicio 4

Si se registra dos veces el id `2`, ¿qué ocurre? ¿Qué devuelve eliminar `999` y qué datos cambia?

6.7.5 Ejercicio 5

Elige entre `$$campo` y un arreglo/Dictionary para una ciudad que llega de un formulario. Justifica la elección.

6.7.6 Respuestas

6.7.7 Respuesta 1

Volver al ejercicio. El cast toma el prefijo `125` y descarta que el resto haga inválido el formato; primero debe verificarse que todo el texto cumpla la regla del id.

6.7.8 Respuesta 2

Volver al ejercicio. El resultado es `33.75`. Con `"3"`, el modo estricto de la llamada produce `TypeError`; el contrato detecta la discrepancia, pero no valida por sí solo una entrada web.

6.7.9 Respuesta 3

Volver al ejercicio. La primera es verdadera por la comparación numérica flexible; la segunda es falsa porque también exige que ambos sean el mismo texto y del mismo tipo.

6.7.10 Respuesta 4

Volver al ejercicio. El segundo registro reemplaza al valor de la clave `2`. Eliminar `999` devuelve `false` y no modifica la colección.

6.7.11 Respuesta 5

Volver al ejercicio. El arreglo o `Dictionary` mantiene las claves visibles, permite recorrerlas y evita crear nombres de variables desde una entrada externa; `$$campo` queda para un caso controlado y explícito.

7. Semana 2 · Clase 2: formularios, GET y POST

Un formulario convierte una elección del navegador en una solicitud HTTP. Esta clase sigue el recorrido completo: construir la entrada, leerla con cuidado, validarla en el servidor y mostrar un resultado sin repetir un registro. Los ejemplos son prácticas locales, con datos ficticios y sin base de datos.

El alcance sigue el material de clase aportado como `semana2-clase2.md`, contrastado con las prácticas locales de PHP y C#; no implica acceso nuevo a Blackboard [14].

7.1 Formularios: contrato de entrada

`action` indica el destino y `method` el método HTTP. Cada control que deba viajar necesita `name`: ese nombre será la clave recibida. `enctype="multipart/form-data"` se usa cuando hay archivos; entonces estos llegan por `$_FILES` en PHP. `required`, `type="email"`, `min` y `maxlength` ayudan al navegador, pero una persona puede cambiar una petición antes de enviarla. Por ello son una ayuda de interfaz, no una validación del servidor.

```
<form action="semana2-registro.php" method="post">
  <input name="nombre" required maxlength="80">
  <input name="correo" type="email" required>
  <input name="edad" type="number" min="16" max="99">
        <select    name="carrera"><option    value="SOF">Software</option><option
value="SIS">Sistemas</option></select>
  <label><input name="intereses[]" type="checkbox" value="web">Web</label>
  <button>Registrar</button>
</form>
```

Predicción: el navegador impedirá normalmente enviar nombre o correo vacíos, pero un POST construido fuera del formulario puede no respetar esas reglas. **Comprobación en la práctica:** PHP y C# vuelven a validar en el servidor; una entrada inválida responde `422` y no crea un registro.

PHP usa `intereses[]` para que las casillas se reciban como arreglo. La vista Razor usa varias casillas con el mismo `name="Intereses"`; el *model binder* recibe los valores repetidos como `string[]`. En ambos casos la lista permitida es `web`, `movil` y `datos`; ninguna casilla es obligatoria.

7.2 GET y POST expresan intenciones distintas

GET pone parámetros en la URL, por ejemplo `?q=ana`, y sirve para consultar, buscar o filtrar. Esa URL se puede copiar, marcar o compartir cuando sus datos no son sensibles. POST transporta campos en el cuerpo de la petición y se usa para crear, modificar o eliminar. Que POST no aparezca en la barra de direcciones no significa que esté cifrado: la confidencialidad de datos reales depende de HTTPS.

Decisión	GET	POST
Ubicación	URL y cadena de consulta	cuerpo HTTP
Intención de esta práctica	buscar	registrar
Resultado compartible	sí, si no contiene información sensible	no como enlace de consulta
Cifrado	necesita HTTPS	también necesita HTTPS

No uses `$_REQUEST` para decidir de dónde vino un dato: combina GET, POST y cookies, por lo que oculta una decisión importante. PHP separa `$_GET`, `$_POST`, `$_FILES` y `$_SERVER`. La aplicación C# expresa la misma separación con una acción GET para buscar y parámetros `[FromForm]` en las acciones POST.

```php
$consulta = $_GET['q'] ?? '';
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit('Acceso no válido');
}
```

Predicción: `semana2.php?q=ana` solo filtra lo que ya está en la sesión. **Observación prevista:** después de registrar a `ana@example.test`, buscar `ana88@example.test` mantiene el total de clientes aunque no muestre resultados. El total responde «¿cuántos hay?»; «sin coincidencias» responde otra pregunta.

7.3 Leer, normalizar y validar

Un campo HTTP puede tener una forma inesperada: por ejemplo, `nombre[]` puede llegar como arreglo en vez de texto. Por eso la práctica PHP comprueba primero que `nombre`, `correo`, `edad` y `carrera` sean cadenas y solo después aplica `trim`. En MVC, el *binder* puede representar un valor vacío o espacios como `null`; `Semana2Registro` conserva esas propiedades como `string?` y el controlador normaliza antes de comprobar las reglas.

La normalización no es validación. `trim` quita espacios de borde; no confirma que el nombre sea aceptable. Las reglas del ejercicio son nombre de 1 a 80 caracteres, correo válido, edad opcional pero entre 16 y 99, carrera `SOF` o `SIS`, e intereses de la lista.

Este fragmento se aplica **después** de comprobar que esos campos son cadenas; `$entrada` representa el arreglo de formulario ya recibido. Contar caracteres Unicode no equivale a contar bytes con `strlen`.

```php
$nombre = trim($entrada['nombre'] ?? '');
$longitud = preg_match_all('/./us', $nombre);
if ($longitud === false || $longitud < 1 || $longitud > 80) {
    $errores['nombre'] = 'El nombre es obligatorio (máximo 80 caracteres).';
}
if (filter_var(trim($entrada['correo'] ?? ''), FILTER_VALIDATE_EMAIL) === false) {
    $errores['correo'] = 'Ingrese un correo válido.';
}
```

El material de clase puede mostrar `filter_input(INPUT_POST, ...)`. La fuente de esta práctica valida el arreglo ya recibido con `filter_var`, tras verificar su forma; para un correo escalar ambos aplican la misma regla de validación, pero `filter_var` permite conservar y revisar la entrada normalizada. No conviertas esa equivalencia de regla en una afirmación de que las dos funciones tienen idéntico contexto o comportamiento para toda entrada [1].

```csharp
registro.Nombre = (registro.Nombre ?? string.Empty).Trim();
if (registro.Nombre.Length is < 1 or > 80)
    ModelState.AddModelError(nameof(registro.Nombre),
        "El nombre es obligatorio (máximo 80 caracteres).");
if (Request.Form["Nombre"].Count != 1)
    ModelState.AddModelError("Nombre", "El campo debe ser texto.");
```

Este fragmento resume el comportamiento real de `Semana2Controller`: la comprobación de forma evita tratar una colección como texto; el modelo `Semana2Registro` mantiene `Edad` como texto para poder devolver un valor incorrecto a la persona que debe corregirlo.

Al mostrar datos, PHP usa `htmlspecialchars` con `ENT_QUOTES` y Razor codifica texto de forma predeterminada. Escapar protege la salida HTML; no cambia el valor almacenado ni reemplaza la validación de entrada.

7.4 Dos destinos POST y respuestas HTTP

La práctica ofrece dos variantes deliberadas. «Enviar a otro archivo» usa `POST /semana2-registro.php`, que delega en el manejador de PHP, y `POST /Semana2/Registrar` en MVC. «Procesar en esta página» usa `POST /semana2.php` o `POST /Semana2`. Ambas reutilizan la misma validación y ambas distinguen los siguientes casos:

Caso	PHP	ASP.NET Core MVC
formulario inválido	`422`, valores retenidos, sin escritura	`422`, `ModelState`, sin escritura
token CSRF/antiforgery inválido	`403`	`400`
método no admitido	`405`	`405`
registro válido	`303` a GET	`303` a GET

El token CSRF de PHP se guarda por sesión y se compara antes de validar. Las acciones C# tienen `[ValidateAntiForgeryToken]`. Estos mecanismos reducen envíos forjados desde otro sitio; no sustituyen HTTPS.

Tras un POST válido, el patrón Post/Redirect/Get responde `303` hacia la página GET y muestra una confirmación de una sola vez. **Predicción:** recargar la GET no ejecuta nuevamente el POST. **Observación prevista:** el contador permanece en 1 después de registrar una vez y recargar, en lugar de pasar a 2. PRG evita el reenvío accidental; no resuelve por sí solo simultaneidad, idempotencia de operaciones ni reglas de duplicados.

Los clientes se serializan en la sesión del navegador, no en SQL ni en las facturas del experimento independiente. Otra sesión inicia sin esos registros y cerrar o expirar la sesión puede eliminarlos.

7.5 Pasos de práctica local

1. Desde la raíz del repositorio, ejecuta `php -S 127.0.0.1:8081 -t php/public` y abre http://127.0.0.1:8081/semana2.php.
2. Registra el dato ficticio `ana@example.test`, carrera `SOF`, intereses Web y Datos; predice el `303` y verifica el mensaje en la GET posterior.
3. Busca `ana`; después busca `ana88@example.test`. Distingue total almacenado de resultados visibles y observa que no hay una escritura GET.
4. Envía edad `15`, carrera distinta de `SOF`/`SIS` o un correo inválido. Comprueba `422`, valores retenidos y contador sin cambio.
5. Para MVC, desde la raíz construye primero la imagen y luego inicia el modo sin base de datos. Si ya hay un servidor en `18083`, utiliza ese servidor o detené únicamente tu instancia anterior; no borres volúmenes.

```
docker compose -f csharp/compose.yaml build app
docker compose -f csharp/compose.yaml run --rm --no-deps \\
  -e PracticeOnly=true -p 127.0.0.1:18083:8080 \\
  app dotnet run --no-launch-profile
```

6. Abre http://127.0.0.1:18083/Semana2 y repite la comparación entre la ruta separada `/Semana2/Registrar` y el POST autoprocesado `/Semana2`. `PracticeOnly=true` es el modo aislado de esta práctica: evita la migración inicial de facturas, no convierte el ejercicio en una prueba de SQL.

Consulta primero tipos, conversiones y Cliente y usa la navegación de Aplicaciones Web para volver al mapa de la asignatura.

7.6 Ejercicios

7.6.1 Ejercicio 1: contrato del formulario

¿Por qué `required` no basta para aceptar un correo y qué papel cumple `name`?

7.6.2 Ejercicio 2: elegir método

Elige GET o POST para buscar `ana` y para registrar `ana@example.test`. Explica por qué POST no cifra el correo.

7.6.3 Ejercicio 3: múltiples intereses

Compara `intereses[]` en PHP con varias casillas `name="Intereses"` en Razor. ¿Qué validación se conserva en ambos lados?

7.6.4 Ejercicio 4: entrada manipulada

¿Qué debe pasar si `nombre` llega como arreglo o si la edad es `100`? Incluye estado, datos retenidos y efecto sobre el contador.

7.6.5 Ejercicio 5: PRG y alcance

¿Por qué la redirección `303` ayuda al recargar, y qué problema no resuelve? ¿Dónde viven los registros de esta práctica?

7.7 Respuestas razonadas

7.7.1 Respuesta 1

Volver al ejercicio. El navegador puede omitir o manipular sus controles, así que el servidor valida el correo. `name` fija la clave con la que la solicitud entrega el valor; sin esa clave el control no participa como dato nombrado del formulario.

7.7.2 Respuesta 2

Volver al ejercicio. La búsqueda es GET porque consulta y puede ser una URL compartible; el registro es POST porque cambia la colección de la sesión. POST solo mueve el dato fuera de la URL: HTTPS protege su tránsito.

7.7.3 Respuesta 3

Volver al ejercicio. PHP agrupa con `[]`; MVC agrupa por el mismo nombre repetido. Los dos resultados deben ser colecciones cuyos elementos estén entre `web`, `movil` y `datos`; el servidor no acepta una clave o valor alterado.

7.7.4 Respuesta 4

Volver al ejercicio. La forma arreglo se rechaza antes de hacer `trim`; `100` incumple el rango. La práctica devuelve `422`, conserva valores para corregir y no agrega un cliente, así que el contador no aumenta.

7.7.5 Respuesta 5

Volver al ejercicio. Tras guardar, `303` lleva a GET: actualizar esa página no repite el cuerpo POST. No garantiza idempotencia ni coordina dos envíos simultáneos. Los registros viven solo en la sesión de ese navegador, no en MySQL ni en las facturas [4].

8. Tablas de consulta HTTP

Usa estas tablas al estudiar el ciclo petición/respuesta y al inspeccionar **Network (Red)** del navegador. Son una referencia técnica de apoyo, no una nueva semana del temario. Las rutas de productos son ejemplos didácticos: no son servicios del repositorio.

8.1 Métodos: qué le pides al servidor

El **método** expresa la intención de la petición. La **ruta** identifica el recurso; la API define sus reglas y los formatos que acepta.

Método	Qué significa	Ejemplo de petición
`GET`	Obtener una representación del recurso, sin solicitar su modificación.	`GET /productos`
`POST`	Pedir que el recurso procese los datos enviados; puede crear otro recurso o ejecutar una operación definida por la API.	`POST /productos`
`PUT`	Crear o reemplazar el estado del recurso identificado mediante la representación enviada. No equivale a cambiar solo un campo.	`PUT /productos/42`
`PATCH`	Aplicar cambios parciales según un formato de parche y las reglas de la API.	`PATCH /productos/42`
`DELETE`	Solicitar que se retire la asociación entre la URL y el recurso. No garantiza borrado físico inmediato de archivos o registros.	`DELETE /productos/42`
`HEAD`	Pedir las cabeceras que corresponderían a `GET`, pero sin cuerpo de respuesta.	`HEAD /productos/42`
`OPTIONS`	Consultar opciones de comunicación del destino. También se utiliza en las comprobaciones previas de CORS.	`OPTIONS /productos/42`

PUT frente a PATCH. Para un producto con nombre y precio, `PUT` envía su representación de reemplazo conforme al contrato; un parche puede cambiar solo el precio. No basta con cambiar el verbo: el servidor debe implementar la operación y validar el formato. `POST` no significa exclusivamente «crear».

8.2 Seguridad semántica e idempotencia

Seguro, en HTTP, significa que el cliente no solicita cambiar el estado del recurso; no significa que haya cifrado, autenticación o permisos. **Idempotente** significa que repetir la misma petición tiene el mismo efecto pretendido sobre el servidor que enviarla una vez, no que devuelva siempre el mismo cuerpo o estado.

Método	¿Seguro según HTTP?	¿Idempotente por su semántica?
`GET`, `HEAD`, `OPTIONS`	Sí	Sí
`PUT`, `DELETE`	No	Sí
`POST`, `PATCH`	No	No está garantizado

Por ejemplo, un primer `DELETE` puede responder `204` y el siguiente `404`: el recurso sigue ausente y la operación puede seguir siendo idempotente. Un `PATCH` que fija un precio puede ser idempotente, pero uno que lo incrementa no lo es. No repitas automáticamente un `POST` ante una duda: podría duplicar una operación. Practica modificaciones y borrados solo en un simulador local o con datos desechables.

8.3 Dónde van los datos y dónde verlos

Dato	Dónde buscar en DevTools	Interpretación en PHP
Método y URL	**Headers → General**: Request Method y Request URL.	`$_SERVER['REQUEST_METHOD']` identifica el verbo.
Query, como `?categoria=libros`	URL y **Payload → Query String Parameters**, cuando aparezca esa sección.	`$_GET` contiene los parámetros de la URL, incluso si el método es `POST` u otro.
Formulario enviado por `POST`	**Payload → Form Data**.	`$_POST` para los tipos de formulario que PHP procesa, no para cualquier cuerpo.
JSON enviado por `POST`, `PUT` o `PATCH`	**Payload**, como contenido enviado; comprueba también `Content-Type`.	Leer `php://input`, decodificar y validar el JSON según el contrato. No esperar que aparezca en `$_POST`.
Cabeceras de la petición	**Headers → Request Headers**.	`Content-Type` describe el formato enviado; `Accept`, el formato de respuesta preferido.
Resultado del servidor	Estado en **Headers → General**; contenido en **Response**.	El programa emite un estado, cabeceras y, cuando corresponde, un cuerpo.

La query puede acompañar distintos métodos: **no es exclusiva de GET**. En `fetch()` del navegador no envíes cuerpo con `GET` ni `HEAD`. En `DELETE`, no supongas que un cuerpo tiene un significado estándar: sigue el contrato de la API. Los nombres y la organización de las pestañas pueden variar entre navegadores.

8.4 Estados: qué respondió el servidor

Un método no es un estado: `POST` describe lo solicitado; `201` describe su resultado. Ningún verbo garantiza por sí solo un código de éxito.

Estado	Qué significa	Qué comprobar
`200 OK`	Petición atendida con éxito.	Leer el contenido; no deducir que es JSON solo por el código.
`201 Created`	Se creó uno o más recursos.	Revisar el recurso devuelto y `Location`, si se proporciona.
`204 No Content`	Éxito sin cuerpo de respuesta.	No intentar interpretar un JSON que no existe.
`400 Bad Request`	El servidor considera incorrecta la petición.	Sintaxis, parámetros y mensaje recibido.

Estado	Qué significa	Qué comprobar
`401 Unauthorized`	Faltan credenciales de autenticación válidas.	No confundir autenticación con permisos.
`403 Forbidden`	El servidor entiende la petición, pero se niega a atenderla.	Permisos y políticas; identificarse no garantiza acceso.
`404 Not Found`	No se encontró el recurso o el servidor no revela su existencia.	URL e identificador; no implica por sí solo un fallo del programa.
`405 Method Not Allowed`	Ese recurso no admite el método solicitado.	Cabecera `Allow` y documentación de la ruta.
`415 Unsupported Media Type`	No se acepta el formato del contenido enviado.	`Content-Type` y formato del cuerpo.
`500 Internal Server Error`	Fallo interno al procesar la petición.	Evidencia del servidor; no atribuirlo automáticamente al navegador.

Lectura razonada. Si ves `PATCH /productos/42` con estado `415`, el verbo es `PATCH`, no `415`. Compara el formato de parche esperado con el `Content-Type` y el cuerpo que realmente enviaste antes de cambiar la URL o repetir la petición. Si una respuesta `HEAD` no muestra cuerpo, no lo trates como un error: es lo esperado.

8.5 Fuentes de consulta

Referencia técnica: MDN: métodos HTTP, MDN: estados HTTP, HTTP Semantics, RFC 9110 y PATCH, RFC 5789. Para entradas PHP, consulta el manual de variables externas. Practica las cinco peticiones principales y su inspección real en Laboratorio PHP: cinco métodos en DevTools.

9. Laboratorio PHP: cinco métodos en DevTools

Este laboratorio transversal permite observar peticiones reales a un servidor PHP local. Practicarás `GET`, `POST`, `PUT`, `PATCH` y `DELETE` sin base de datos ni persistencia: todos los datos son ficticios y **DELETE solo simula** una eliminación.

9.1 Ruta rápida

Necesitas PHP instalado. Desde la raíz del repositorio ejecuta:

```
php -S 127.0.0.1:8088 -t ebook/examples/http-methods ebook/examples/http-methods/
router.php
```

Después abre el laboratorio local. La guía del ejemplo conserva el mismo comando y sirve como referencia descargable del código fuente.

> ⚠ Advertencia
>
> Usa exactamente `127.0.0.1`, el document root aislado `ebook/examples/http-methods` y `router.php`, que limita las rutas públicas. No omitas el router, no uses `0.0.0.0` ni sirvas la raíz del repositorio: podrías exponer otros archivos. El ejemplo requiere PHP local; no funciona con `file://` ni ejecutado desde GitHub Pages.

9.2 Abrir y preparar Network

1. Abre DevTools con `F12`, `Ctrl+Shift+I` o el menú del navegador.
2. Selecciona **Network** o **Red**. Si no aparece, búscala en ».
3. Filtra por `api.php`. Activa **Preserve log / Conservar registro** si quieres mantener las cinco filas.
4. Pulsa un botón del laboratorio y selecciona la nueva fila.
5. Revisa **Headers**, **Payload** y **Response**. Los nombres pueden variar ligeramente según la versión o el idioma del navegador.

En **Headers → General** identifica siempre **Request URL**, **Request Method** y **Status Code**. Luego relaciona lo observado con la fuente que PHP leyó y con el JSON de respuesta.

9.3 GET: parámetros en la URL

Pulsa **GET con query**. En la URL aparecen `tema=red` y `nivel=basico`; DevTools también puede mostrarlos como **Query String Parameters**. PHP los recibe en `$_GET`.

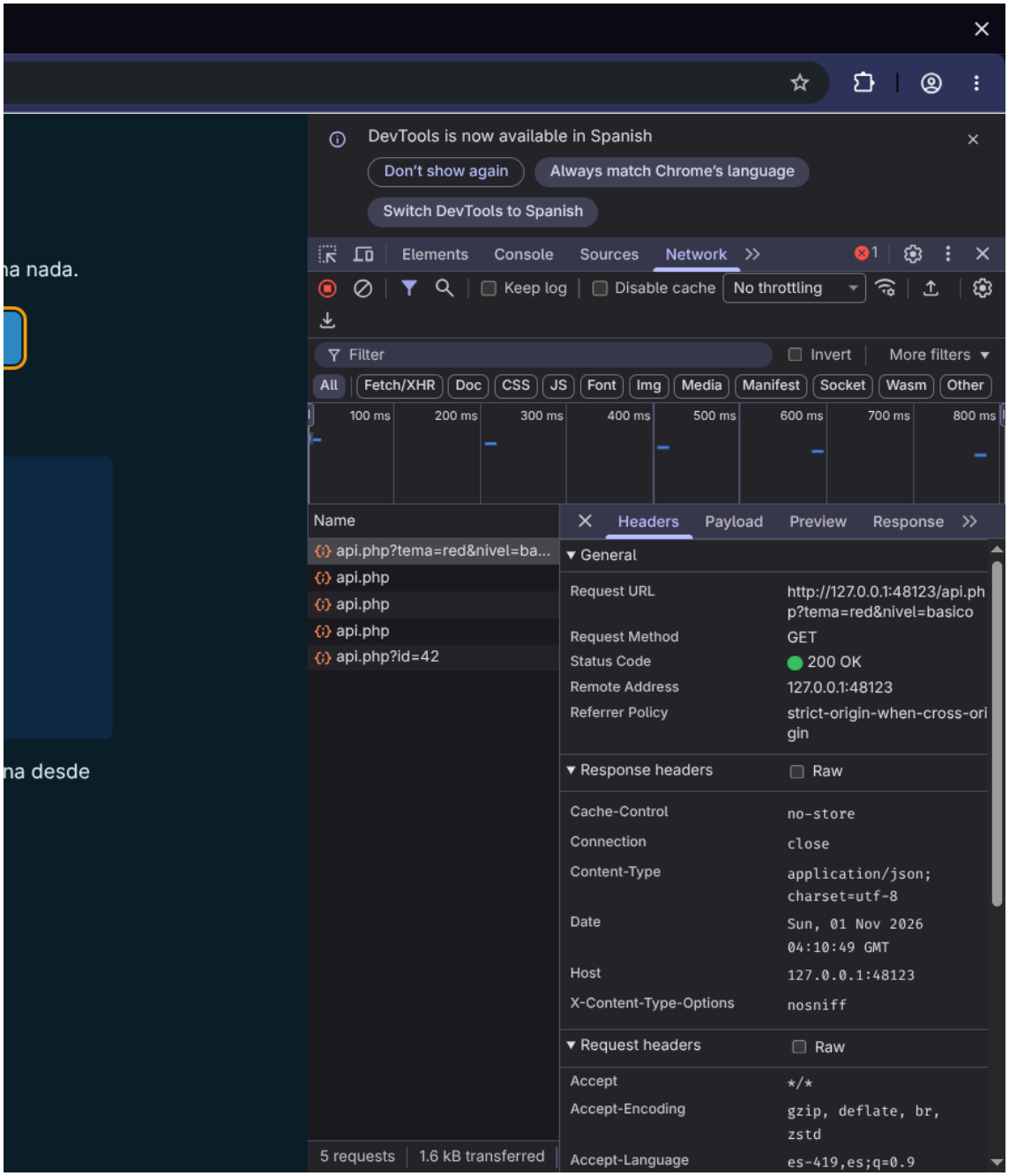


Figura 9.1: DevTools Network unido al laboratorio PHP real: la fila GET seleccionada muestra Request URL, Request Method GET, Status Code 200 y la dirección loopback.

9.4 POST: formulario

Pulsa **POST formulario** y abre **Payload**. El navegador envía `application/x-www-form-urlencoded`; por eso DevTools presenta **Form data** y PHP llena `$_POST`. La query, si existiera, seguiría perteneciendo a `$_GET`.

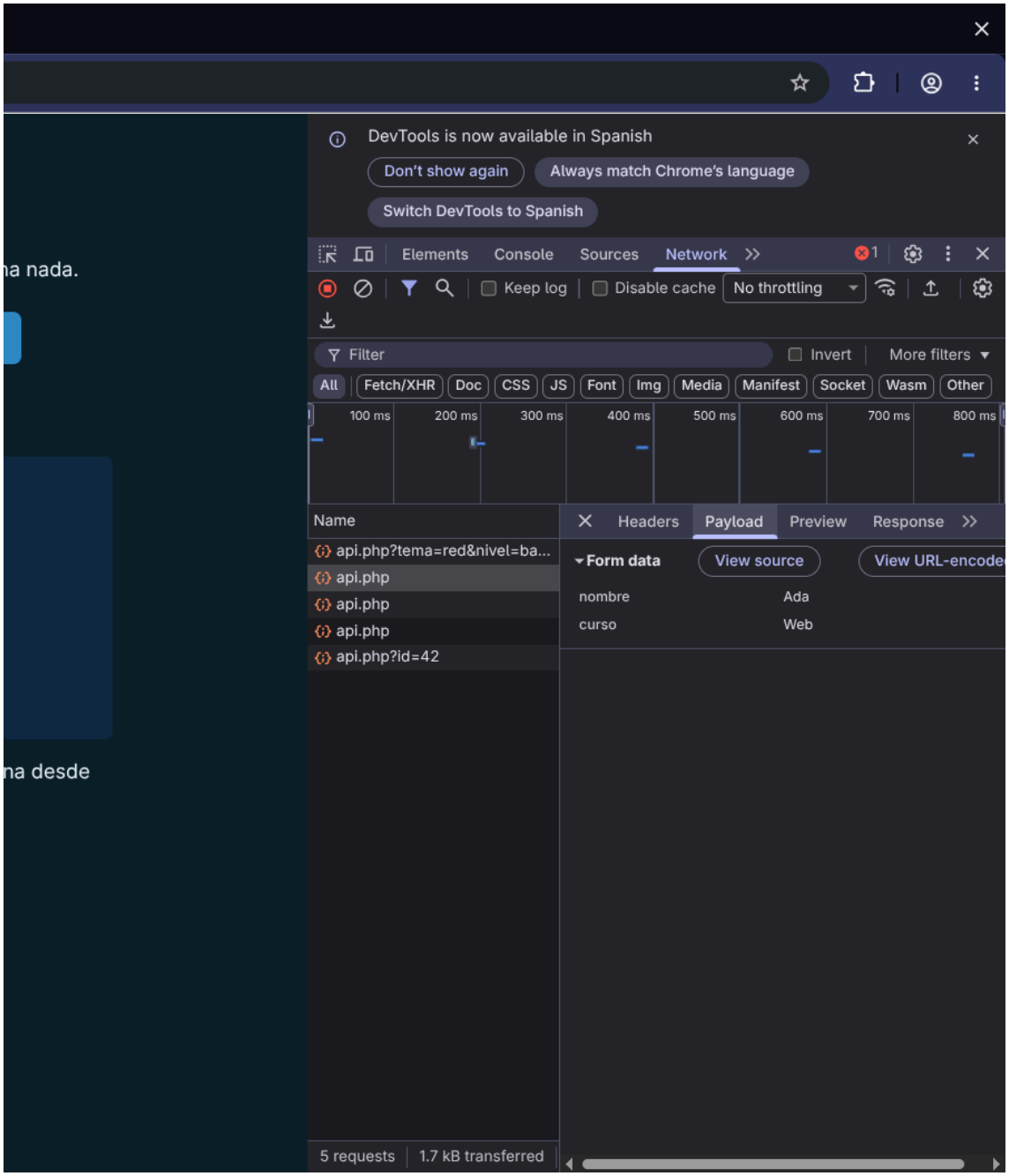


Figura 9.2: DevTools Network unido al laboratorio PHP real: Payload de POST muestra Form data con nombre Ada y curso Web.

9.5 PUT: cuerpo JSON completo de ejemplo

Pulsa **PUT JSON**. DevTools muestra el **Request payload** `{id: 42, nombre: "Libro"}`. PHP no coloca este JSON en `$_POST`: el endpoint lee `php://input` y luego usa `json_decode`.

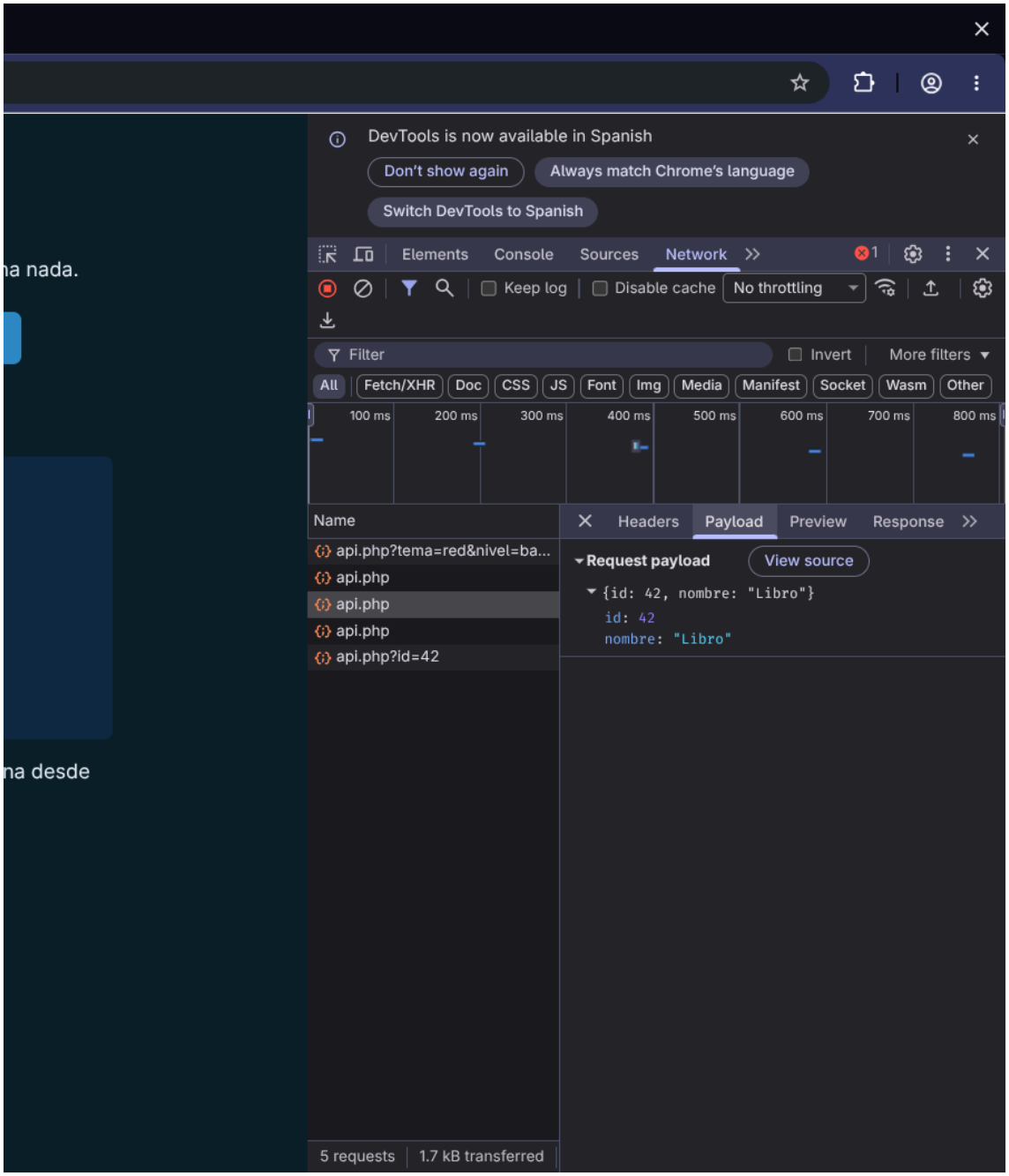


Figura 9.3: DevTools Network unido al laboratorio PHP real: Payload de PUT muestra el JSON con id 42 y nombre Libro.

9.6 PATCH: cambio parcial

Pulsa **PATCH JSON**. El cuerpo contiene solo el identificador y el precio de ejemplo. En **Response**, PHP confirma el método, la fuente `php://input + json_decode`, los datos recibidos y `saved: false`: observar una respuesta correcta no implica que se haya guardado algo.

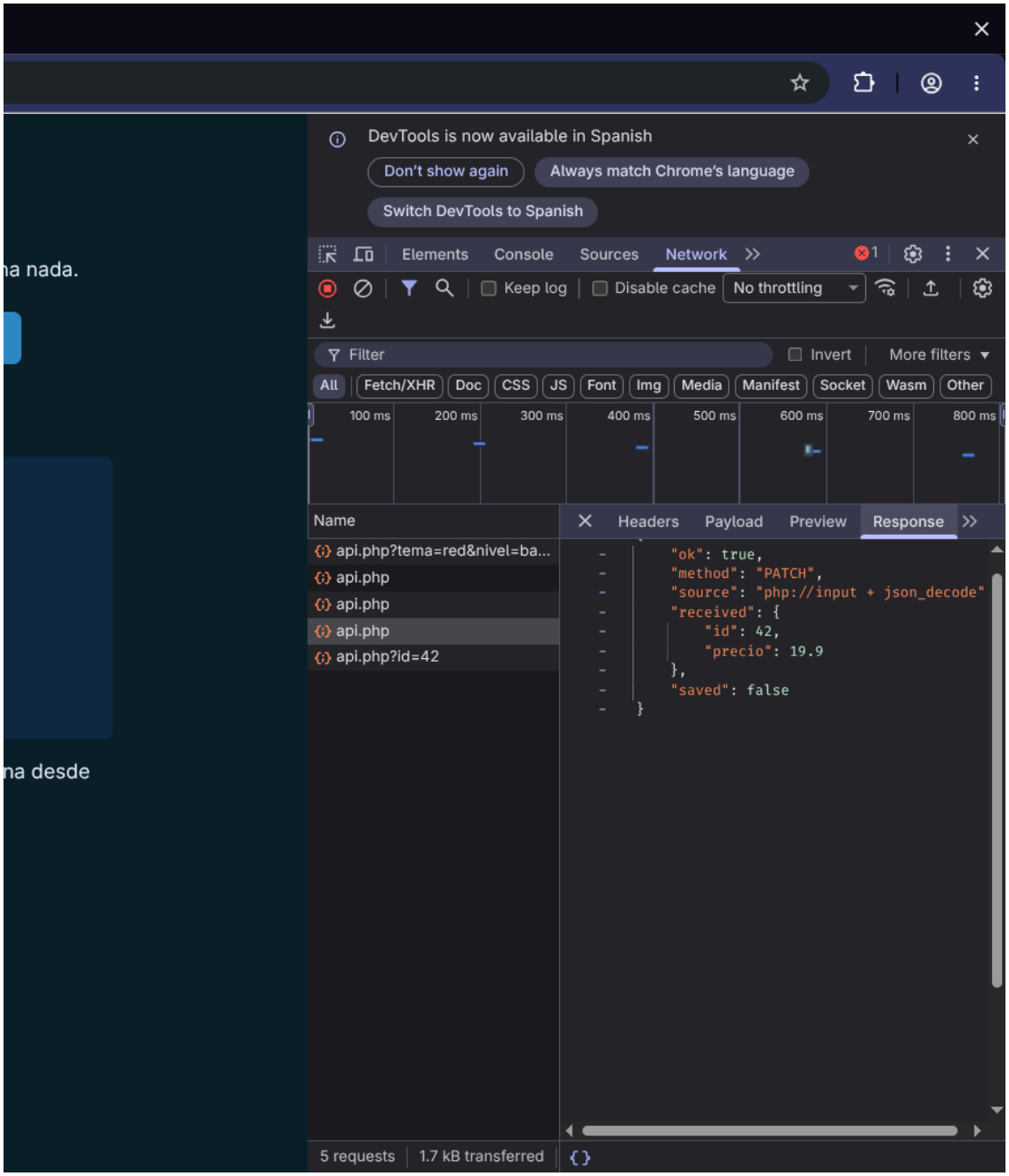


Figura 9.4: DevTools Network unido al laboratorio PHP real: Response de PATCH muestra method PATCH, el origen php input más json decode, precio 19.9 y saved false.

9.7 DELETE: simulación explícita

Pulsa **DELETE simulado**. El identificador viaja como query y PHP lo obtiene de `$_GET`. La respuesta dice `saved: false` y `simulation: "No se eliminó ningún dato."`; no existe archivo, registro o contador que borrar.

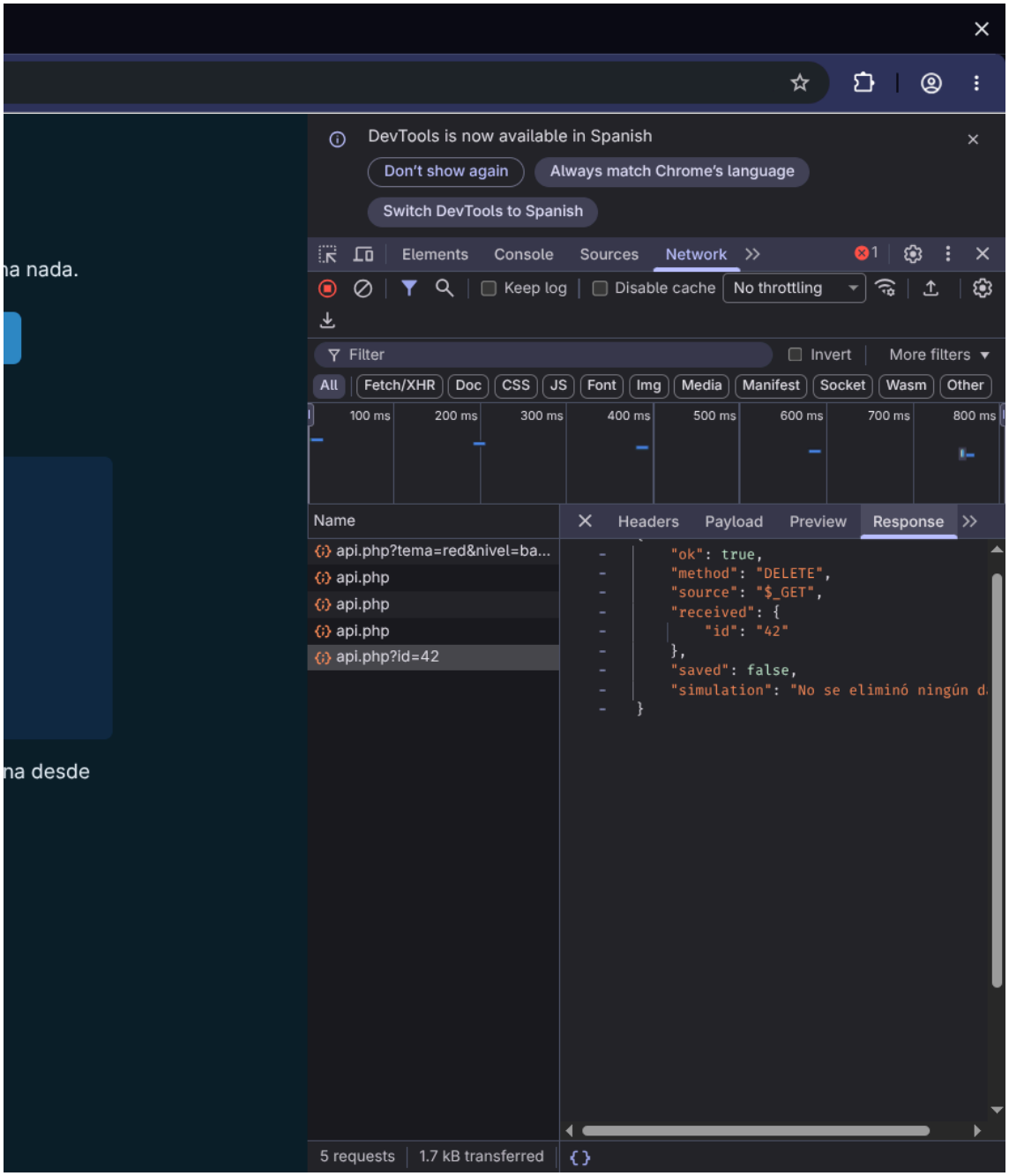


Figura 9.5: DevTools Network unido al laboratorio PHP real: Response de DELETE muestra method DELETE, id 42, saved false y el mensaje de simulación sin eliminación.

9.8 Qué debes poder explicar

Observación	Lectura razonada
URL, método y estado	Identifican el destino, la intención de la petición y el resultado HTTP.
Query String Parameters	Se originan en la URL y llegan a `$_GET`, no solo cuando el método es GET.

Observación	Lectura razonada
Form data de POST	PHP lo procesa en `$_POST` para el tipo de formulario usado.
Request payload JSON	PHP debe leer `php://input`, decodificar y validar el formato.
Response	Es el cuerpo que emitió PHP; aquí refleja entradas ficticias sin guardarlas.

Detén el servidor con `Ctrl+C` al terminar. Para recordar significados, idempotencia y estados, vuelve a Tablas de consulta HTTP.

10. Respuestas de autoevaluación

Estas respuestas muestran una forma de razonar. Compáralas con tus notas después de contestar las preguntas de auto evaluación de Semana 1. No asignan puntos, no guardan progreso y no certifican una evaluación.

Para Semana 2, las soluciones están al final de cada clase: tipos y Cliente y formularios GET/POST.

10.1 P1: historia y propósito

Una página estática entrega contenido ya preparado. Una dinámica puede construir parte de la respuesta con datos de la solicitud, por ejemplo una operación seleccionada. Para hablar de la evolución de PHP verificaría la versión y sus cambios en las notas de publicación oficiales, no en un recuerdo o tutorial sin fecha. Volver a P1

10.2 P2: HTTP

La solicitud es lo que el navegador envía al servidor, incluidos los datos que la calculadora necesita. El procesamiento aplica una regla a esos datos. La respuesta es lo que el servidor devuelve para que el navegador lo muestre. No es necesario confundir una solicitud HTTP con el HTML que finalmente se ve. Volver a P2

10.3 P3: tipos

En PHP, `"8" + "2"` produce el entero `10`: el operador `+` suma y convierte estas dos cadenas numéricas enteras a `int`. En C#, `8 + 2` también produce el entero `10`, pero los operandos ya son `int`. En estos casos coincide el resultado; difieren los tipos de entrada y la conversión que realiza PHP. No lo confundas con la concatenación: en C#, `"8" + "2"` produce `"82"`. Identifica los tipos antes de comparar expresiones [1], [2]. Volver a P3

10.4 P4: interpolación

Los dos fragmentos pretenden producir «Ana tiene 2 intentos.». En PHP las variables aparecen como `$nombre` y `$intentos` dentro de comillas dobles. En C#, el prefijo `$` se aplica a la cadena y las expresiones van entre llaves. Una salida semejante no significa que la sintaxis sea idéntica. Volver a P4

10.5 P5: constantes

Elegiría una constante para una regla que el programa no debe reasignar, como una tasa de ejemplo durante esa ejecución. Elegiría una variable para una entrada que cambia, como `base` o `baseImponible`. La elección debe explicar el rol del dato; no es una decoración del código. Volver a P5

10.6 P6: alcance

Un parámetro aparece en la firma de la función y en cada llamada, por lo que el lector puede reconocer de dónde llega el dato. `global` depende de un nombre externo y oculta esa relación. Pasar parámetros no resuelve todos los diseños, pero es una opción clara para el ejemplo de Semana 1. Volver a P6

10.7 P7: depuración

Anotaría (1) los valores y tipos que llegaron, incluido el divisor `0`; (2) el operador y la rama de procesamiento que se ejecutó; y (3) el resultado o aviso exacto mostrado. Con esas observaciones puedo formular una hipótesis sin atribuir el problema automáticamente al navegador, PHP o C#. Volver a P7

10.8 P8: comprobación

Una tabla suficiente tiene columnas para entrada A, operación, entrada B, predicción, resultado observado y explicación. Incluye `8+2`, `8-2`, `8*2`, `8/2`, un divisor `0` y una entrada no numérica. Las últimas filas documentan el comportamiento declarado: no exigen construir aquí un sistema nuevo de validación. Volver a P8

10.9 P9: MVC

Afirma que MVC es la forma elegida para organizar las prácticas web que lo usan. No afirma que todos los archivos existentes ya estén separados en modelo, vista y controlador. El `php/public/home.php` de facturas sigue siendo monolítico y este libro no lo modifica; además, las facturas son un experimento adicional y no una prueba de requisito de Semana 1. Volver a P9

10.10 P10: transferencia

Para Celsius a Fahrenheit, la entrada es una temperatura Celsius, el procesamiento puede ser `C * 9 / 5 + 32`, y la salida es la temperatura convertida con su unidad. En PHP predeciría que una entrada numérica concreta, por ejemplo `0`, produce `32`; en C# haría la misma predicción antes de correr un fragmento con un tipo numérico apropiado. La comprobación compara la regla y el valor observado, no solo que el programa compile. Volver a P10

10.11 Cómo usar una discrepancia

Si tu respuesta no coincide, vuelve a la pregunta enlazada, localiza qué suposición hiciste sobre entrada, tipo o alcance y prueba un caso pequeño. Consultar una pista o estas soluciones puede orientar la siguiente prueba, pero no es evidencia de que la hayas realizado.

II Diseño y Arquitectura de Software

13 Práctica integrada y autoevaluación de la Semana 1 69
13.1 Ruta de trabajo 69
13.2 Caso guiado: una reserva sin promesas duplicadas 69
13.3 Ejercicio análogo: préstamo de equipos 70
13.4 Autoevaluación formativa 71
13.5 Cierre 74

14 Procesos de software: actividades, flujo y trabajo reutilizable 75
14.1 Objetivos de aprendizaje 75
14.2 Un proceso organiza el trabajo, no un papel: criterio 75
14.3 Flujo: hacer visible cómo avanza el trabajo 75
14.4 Tareas y patrones: distinto nivel de reutilización . 77
14.5 Caso guiado: primer flujo para reservar salas . . 77
14.6 Práctica breve: del trabajo difuso a tareas revisables 78
14.7 Cierre 78

15 Modelos de proceso: comparación, práctica y autoevaluación 79
15.1 Ruta de estudio 79
15.2 Cuatro modelos, cuatro énfasis 79
15.3 Cascada: previsibilidad cuando el cambio es limitado 80
15.4 Incremental: entregar capacidades completas por partes 80
15.5 Evolutivo: aprender con versiones y retroalimentación 80
15.6 Espiral: ciclos guiados por riesgo 80
15.7 Caso trabajado: reservas para una red de bibliotecas 80
15.8 Práctica análoga: agenda de espacios comunitarios 81
15.9 Autoevaluación formativa 82
15.10 Alcance de la fuente y cierre 84

16 Gestión Empresarial y Emprendimiento 85
16.1 Cobertura verificada 85
16.2 Ruta de lectura 85

17 Inteligencia Artificial 87
17.1 Cobertura verificada 87
17.2 Ruta de lectura 87

18 Inteligencia de Negocios 89
18.1 Cobertura verificada 89
18.2 Ruta de lectura 89

19 Proyectos Informáticos 91
19.1 Cobertura verificada 91
19.2 Ruta de lectura 91

20 Fuentes y política de uso 93
20.1 Política de fuentes 93
20.2 Bibliografía 93

20.2 Bibliografía 95

Esta asignatura incorpora las **Semanas 1 y 2** a partir de contenidos oficiales verificados. El recorrido desarrolla explicaciones y casos originales para justificar decisiones de arquitectura y de proceso; no sustituye la clase, no resuelve tareas institucionales ni registra calificaciones [15], [16].

Cobertura actual

Semana	Tema y propósito	Lectura
1	Del análisis a las decisiones de diseño y arquitectura	Diseño y arquitectura: de la necesidad a las decisiones
1	Tipos, dominios de uso y mitos del software	Software: tipos, dominios y mitos
1	Caso integrado, práctica y autoevaluación formativa	Práctica integrada y autoevaluación
2	Actividades generales, flujo, tareas y patrones reutilizables	Procesos de software: actividades, flujo y trabajo reutilizable
2	Cascada, incremental, evolutivo y espiral; práctica y autoevaluación	Modelos de proceso: comparación, práctica y autoevaluación

Alcance de la cobertura actual

La Semana 1 conecta necesidades, atributos de calidad, módulos y estilos; también clasifica software y analiza mitos antes de una práctica integrada. La Semana 2 explica cómo requisitos, diseño, implementación, pruebas y mantenimiento pueden organizarse en distintos flujos. Después compara cascada, incremental, evolutivo y espiral según estabilidad, entregas, retroalimentación y riesgos, con un caso original de reservas de biblioteca y diez preguntas razonadas.

La cobertura de Semana 2 procede del documento HTML oficial verificado. El plan y otros archivos PDF quedaron sin cuerpo legible, por lo que no se afirma que estén cubiertas todas las fuentes, clases o evaluaciones de la semana. Todo el material es formativo para estudiar de manera autónoma, no una tarea nueva ni una calificación.

El enlace oficial también propone un ejercicio formativo en Educaplay. Es un recurso para contrastar ideas: este cuaderno no afirma que se haya completado ni asigna una nota.

Ruta de estudio

1. Formula una respuesta antes de abrir la retroalimentación.
2. Relaciona una necesidad concreta con una decisión y una comprobación observable.
3. Distingue el concepto técnico de su dominio de uso antes de clasificarlo.
4. Explica qué evidencia faltaría para sostener una conclusión arquitectónica.

La cobertura se ampliará únicamente cuando haya temas oficiales verificados. Las prácticas y sus respuestas de Arquitectura viven en las lecturas de cada semana; las respuestas de Aplicaciones Web siguen en su página propia.

11. Diseño y arquitectura: de la necesidad a las decisiones

Meta. Al terminar podrás pasar de una necesidad a una decisión de diseño, explicar por qué importa y proponer cómo observar su resultado. La arquitectura no empieza al dibujar cajas: empieza al hacer explícitas decisiones que condicionan cómo el sistema responderá a cambios y cualidades relevantes [15].

11.1 Objetivos de aprendizaje

- Distinguir análisis, diseño y arquitectura sin reducirlos a una escala de tamaño.
- Usar abstracción, modularidad, encapsulación, acoplamiento y cohesión al razonar.
- Reconocer vistas, relaciones, estilos y atributos de calidad iniciales.

Pregunta de recuperación. Si dos módulos cambian juntos cada vez que cambia una regla, ¿qué hipótesis harías sobre su frontera y por qué?

11.2 De entender la necesidad a decidir una forma

El **análisis** pregunta qué problema, actores, reglas y resultados importan: por ejemplo, una biblioteca necesita reservar ejemplares sin prometer dos veces el mismo libro. El **diseño** decide cómo organizar una solución a ese problema. La **arquitectura** reúne las decisiones de diseño de alto impacto —componentes, interfaces, datos, despliegue y restricciones— y su justificación.

Hay solapamiento: una decisión puede ser arquitectónica en un contexto y local en otro. No existe un corte correcto de «muchas líneas = arquitectura». Elegir una interfaz que consumen varios equipos suele tener más alcance que renombrar una función, aunque la primera sea breve. La pregunta útil es qué opciones habilita o cierra la decisión y quién deberá convivir con ella.

11.3 Separar para comprender y cambiar

Una **abstracción** conserva lo esencial y oculta detalle accidental: `Reserva` representa una intención, no cada tabla de la base de datos. La **modularidad** agrupa responsabilidades para que puedan comprenderse y modificarse con cierta independencia. La **encapsulación** protege el estado y las reglas detrás de una interfaz; no es solo declarar campos privados.

Dos medidas cualitativas orientan la conversación. El **acoplamiento** es cuánto necesita saber o depender una parte de otra: se prefiere bajo cuando evita cambios en cascada. La **cohesión** expresa cuán relacionadas están las tareas dentro de un módulo: se prefiere alta cuando una razón clara explica por qué viven juntas. Son objetivos de diseño, no números mágicos. La ocultación de decisiones propensas a cambiar es una razón histórica para fijarse en esas fronteras [17].

El diseño **estructurado** suele descomponer por procesos y transformaciones de datos; el **orientado a objetos** organiza colaboraciones entre objetos con estado y comportamiento. Ninguno anula el otro: un sistema OO también tiene flujos y datos, y uno estructurado puede tener módulos bien delimitados. Se elige la representación que hace más visible el problema.

11.4 Vistas que responden preguntas distintas

Vista	Pregunta que aclara	Ejemplo en reservas
Conceptual	¿Qué conceptos y reglas existen?	ejemplar, persona, reserva y disponibilidad
Lógica	¿Qué componentes colaboran?	catálogo, reservas y notificaciones

Vista	Pregunta que aclara	Ejemplo en reservas
Física	¿Dónde se ejecutan y guardan?	navegador, servicio y base de datos

Un diagrama de **casos de uso** sitúa a la persona usuaria y metas como «reservar»; un diagrama de **clases** propone conceptos, atributos y relaciones; uno de **secuencia** muestra mensajes en un escenario. Son notaciones para comunicar, no una garantía de que el diseño sea bueno. Un patrón nombra una solución recurrente con sus consecuencias; la documentación registra contexto, decisión, alternativas y razón para que no haya que adivinarla después.

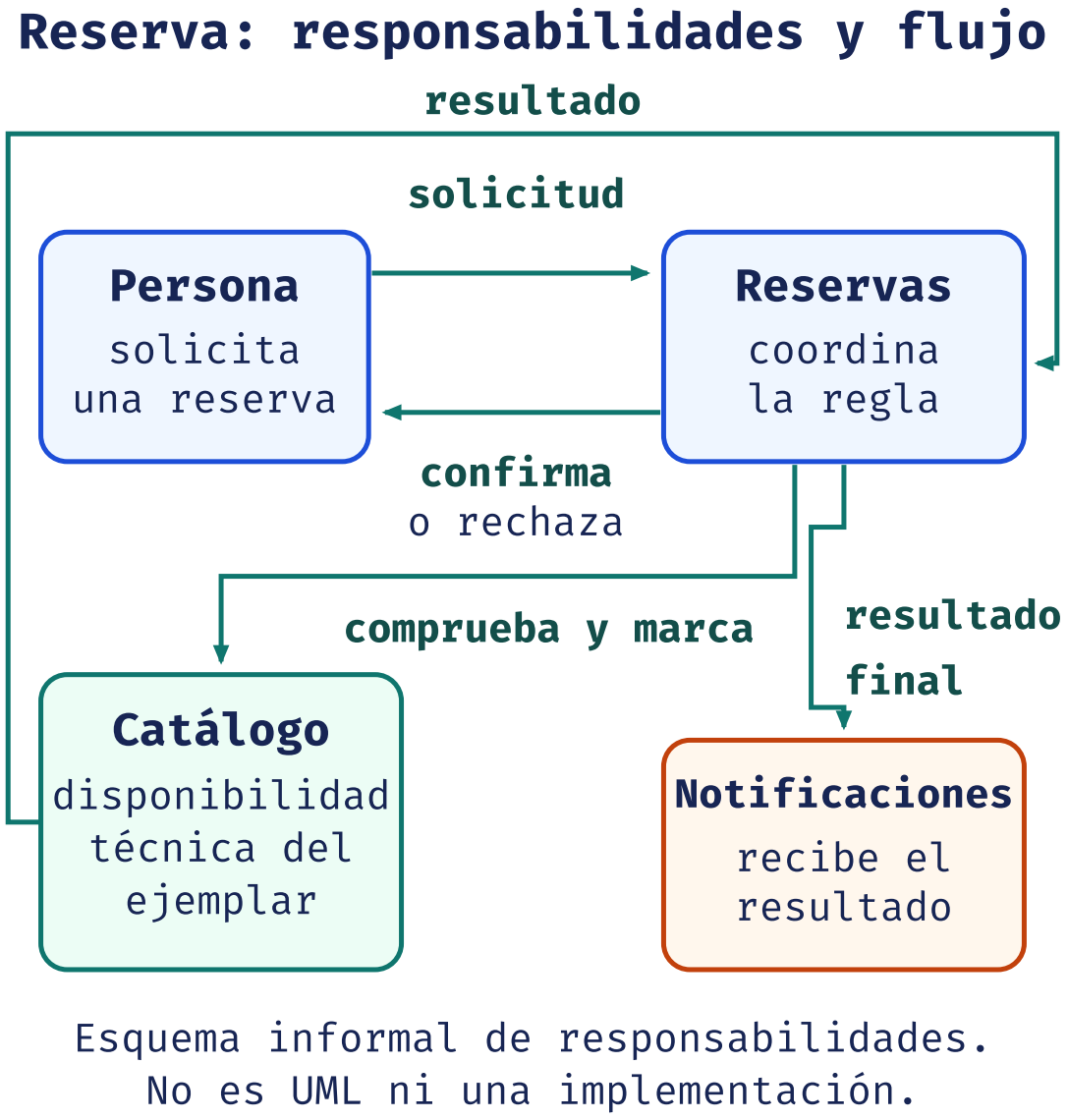


Figura 11.1: Mapa conceptual de una reserva: la persona solicita una reserva; el módulo Reservas consulta Catálogo y comunica el resultado.

Nota. El mapa es un esquema informal de responsabilidades e interfaces, no un diagrama UML formal.

11.5 Caso guiado: reservar sin duplicar una promesa

Necesidad de calidad. Cuando dos personas intentan reservar el mismo ejemplar, el sistema debe evitar confirmar ambas reservas. La calidad aquí es consistencia de la operación y la confiabilidad de la información. No debe confundirse la disponibilidad de un ejemplar con la disponibilidad del servicio; no es un benchmark medido.

Decisión. El módulo `Reservas` expone `solicitar(ejemplar, persona)` y delega a la interfaz `Disponibilidad` una operación atómica de comprobar y marcar. El módulo `Notificaciones` recibe solo el resultado final. Así la regla de no duplicación queda encapsulada, y la interfaz evita que una pantalla modifique el inventario por su cuenta.

Comprobación observable propuesta. En una prueba controlada con dos solicitudes para el mismo ejemplar disponible, se espera una confirmación y un rechazo; el registro debe mostrar una sola reserva activa. Es un criterio hipotético para discutir la decisión, no una rúbrica docente ni una medición de rendimiento.

11.6 Estilos y cualidades: opciones con consecuencias

Cliente-servidor separa quien solicita de quien presta un servicio. Capas organizan, por ejemplo, presentación, reglas y persistencia. Un monolito puede mantener esos límites dentro de un único despliegue; microservicios los distribuyen entre servicios independientes. Se pueden combinar: un monolito por capas puede atender clientes, y un conjunto de servicios también puede usar capas. Microservicios no son un ascenso automático ni una cura para un diseño poco claro.

Atributo	Pregunta inicial	Decisión que podría influir
Rendimiento	¿cuánto tarda una reserva bajo una carga definida?	consulta e índices adecuados
Seguridad	¿quién puede reservar o administrar?	autorización en la interfaz
Escalabilidad	¿qué crece y cómo se distribuye?	separar trabajo repetible si se justifica
Disponibilidad	¿qué ocurre si falla un componente?	manejo explícito de fallos y degradación planificada
Mantenibilidad y reutilización	¿dónde cambia una regla?	módulos cohesivos e interfaces estables

No hay un estilo mejor sin contexto, restricciones y evidencia. Tampoco conviene partir en microservicios antes de conocer las fronteras, la operación y el coste de coordinar despliegues, fallos y datos.

11.7 Ejercicio breve

Una pantalla consulta directamente la base de datos y manda correos de confirmación. Propón una frontera de módulos y explica qué cambio quedaría más localizado.

> **i** Retroalimentación razonada
>
> Una respuesta posible separa la pantalla, `Reservas` y `Notificaciones`; la pantalla pide la operación a una interfaz y no conoce la persistencia. Cambiar el proveedor de correo afectaría principalmente Notificaciones. La propuesta necesita validar qué reglas comparten realmente los casos de uso: separar por moda también puede añadir complejidad.

11.8 Cierre

Diseñar no es dibujar por adelantado todo el sistema: es elegir representaciones y fronteras que permitan explicar una necesidad y comprobar sus consecuencias. La arquitectura hace visibles las decisiones de

mayor alcance; la evolución posterior sigue siendo contextual, no una ley idéntica para todo programa [18].

Recuerdo rápido. ¿Qué convertirías primero en una comprobación observable antes de afirmar que una decisión mejoró la calidad?

12. Software: tipos, dominios y mitos

Meta. Al terminar podrás clasificar un producto por la función técnica que cumple y por el dominio donde se usa, sin confundir herramientas con categorías. También podrás reemplazar afirmaciones absolutas por preguntas que guíen una acción responsable [15].

12.1 Objetivos de aprendizaje

- Distinguir tipos técnicos de software de sus dominios de aplicación.
- Explicar por qué lenguaje, intérprete, editor, framework y runtime no son sinónimos.
- Analizar los cuatro mitos de la semana mediante condiciones, consecuencias y acciones.

Pregunta de recuperación. ¿Puede el mismo programa ser software de aplicación y pertenecer al dominio educativo? Justifica con dos criterios distintos.

12.2 Dos ejes para no mezclar etiquetas

El **tipo técnico** describe el papel que el software cumple dentro de un entorno. El **dominio de uso** describe el problema humano, organizacional o físico al que se aplica. Un sistema puede tener varias piezas técnicas y servir a un dominio concreto; por eso «educativo» no compite con «de aplicación» como si fueran respuestas al mismo enunciado.

Tipo técnico	Papel principal	Ejemplo posible
De sistema	Gestiona recursos o proporciona base para otros programas	sistema operativo o controlador
De aplicación	Ayuda directamente a realizar una tarea de usuario	portal para reservar libros
De desarrollo	Ayuda a crear, probar o entregar otros programas	compilador o depurador

Los dominios dan otro contexto: una aplicación de facturación pertenece al negocio; un aula virtual, a educación; un programa que procesa datos de un experimento, a la ciencia; y el firmware que controla un sensor, a un sistema embebido. Esta etiqueta no determina por sí sola el lenguaje ni la arquitectura.

12.3 Una herramienta no cambia de categoría por su nombre

Python es un **lenguaje**: un conjunto de reglas para expresar programas. Un **intérprete** de Python ejecuta o evalúa esos programas. Un editor ayuda a escribir archivos; un framework aporta convenciones y componentes para construir una clase de aplicación; un runtime ofrece el entorno de ejecución. Cumplen funciones distintas, aunque una distribución los presente juntos.

Por ello, Python, su intérprete y un editor no son tres «tipos principales» de software comparables. Un editor puede ser software de desarrollo; el intérprete puede formar parte de la plataforma que ejecuta un programa; un framework no define el dominio del producto. Preguntar «¿qué hace esta pieza para quién?» evita una clasificación por marcas.

12.4 Clasificación trabajada

Considera una aplicación web donde estudiantes reservan salas de estudio. La interfaz que muestra horarios y confirma reservas es **software de aplicación**: atiende una tarea de quienes usan la biblioteca. Su dominio es **educativo**, porque organiza un recurso de aprendizaje, aunque también tenga reglas administrativas. El sistema operativo del servidor que la sostiene gestiona recursos y es software de **sistema**; el editor y las pruebas automatizadas que usa el equipo son software de **desarrollo**.

Ahora cambia el dominio sin cambiar la tecnología: el mismo lenguaje Python, el mismo framework web y una base de datos pueden sostener reservas de equipos en un laboratorio científico. La aplicación sigue siendo de aplicación, pero el dominio prioritario pasa a ser **científico**. La tecnología describe una elección de implementación; la finalidad y personas atendidas justifican el dominio.

12.5 Cuatro mitos que ocultan condiciones

12.5.1 «Agregar programadores tarde arregla el atraso»

Puede parecer razonable cuando hay tareas pendientes. Falla si quienes llegan deben aprender contexto, coordinarse y dividir un trabajo que no era divisible: al principio puede aumentar la comunicación y los errores. Un contraejemplo es una tarea aislada, bien especificada y con incorporación preparada, que sí puede repartir trabajo útil. La acción no es prohibir contratar: es revisar el cuello de botella, el tiempo de incorporación y qué entregable puede asumir cada persona.

12.5.2 «Si funciona, está terminado»

Una demostración que produce el resultado esperado aporta evidencia, pero no prueba seguridad, mantenibilidad, operación ni comprensión de los límites. Por ejemplo, una reserva puede funcionar hoy y carecer de control de acceso o de registro para atender un incidente. La acción es acordar qué significa terminado para ese contexto: pruebas pertinentes, revisión, documentación y condiciones de entrega, sin suponer una lista idéntica para todo producto.

12.5.3 «Cada cambio es fácil»

Cambiar una línea puede ser sencillo; cambiar una regla compartida, una interfaz o un dato histórico puede afectar contratos y personas usuarias. El contraejemplo es un texto aislado con pruebas que confirma el cambio. Antes de estimar, se rastrean módulos, dependencias y datos; después se prueba el comportamiento afectado. El software puede evolucionar, pero no todo cambio tiene el mismo coste ni riesgo.

12.5.4 «No hace falta documentación»

El código muestra una implementación, no siempre por qué se eligió, qué alternativa se descartó o qué supuesto operativo sigue vigente. En un equipo pequeño y una corrección local, una explicación breve junto al cambio puede bastar; eso no elimina la necesidad de dejar decisiones relevantes recuperables. La acción es documentar lo que ayuda a operar, cambiar y revisar: propósito, interfaz, decisión y evidencia, manteniéndolo cercano a quien lo usa.

12.6 Ejercicio breve

Clasifica un programa que recoge lecturas de temperatura desde un sensor y envía una alarma a personal de mantenimiento. Indica tipo técnico, dominio y una pregunta que harías antes de decidir si es embebido.

> **i** Retroalimentación razonada
>
> El programa que controla directamente el sensor puede ser software de sistema o embebido según su función y entorno; el dominio podría ser industrial o científico, según el propósito de las mediciones. Preguntaría si se ejecuta dentro de un dispositivo con recursos y responsabilidades específicas, en vez de etiquetarlo solo porque usa un sensor. La alarma puede ser otra aplicación conectada: clasificar sus piezas evita forzar una sola etiqueta al conjunto.

12.7 Conclusión

Clasificar con dos ejes da una explicación verificable: primero el papel técnico, después el dominio y la evidencia de su finalidad. Los mitos fallan porque borran condiciones; una decisión madura las vuelve preguntas sobre personas, dependencias, calidad y contexto. Esa conclusión permite actuar sin prometer que toda contratación, cambio o documentación tenga el mismo resultado.

Recuerdo rápido. Ante «funciona, luego está terminado», ¿qué evidencia de calidad pedirías para este producto y por qué?

13. Práctica integrada y autoevaluación de la Semana 1

Propósito. Esta propuesta de estudio integra los conceptos de la Semana 1 para preparar una evaluación: no es una nueva tarea institucional ni un diseño de producción. Trabajarás sobre el caso ficticio de reservas de biblioteca ya usado en la teoría, con decisiones que puedes explicar y comprobar de forma observable.

13.1 Ruta de trabajo

1. Lee el caso y escribe una primera decisión antes de abrir una pista.
2. Compara tu explicación con el ejemplo parcial; no copies su forma sin revisar el contexto.
3. Resuelve el ejercicio análogo con tus propios supuestos y contrasta después la autoevaluación.

> ! Qué se espera en esta práctica
>
> No hay ejecución, nota ni evidencia de rendimiento que producir. Conserva una hoja con una necesidad, clasificaciones, una decisión justificada, un esquema de responsabilidades y un criterio observable. Diseños alternativos son válidos cuando mantienen las reglas y explican sus consecuencias.

13.2 Caso guiado: una reserva sin promesas duplicadas

Una biblioteca permite que estudiantes soliciten un ejemplar. Dos solicitudes pueden llegar cuando queda una sola copia. El equipo quiere confirmar únicamente una reserva activa y avisar el resultado. La pantalla actual consulta los datos y envía correos por su cuenta.

13.2.1 1. Del enunciado a la clasificación

Pregunta	Respuesta de trabajo	Por qué importa
Necesidad	No confirmar dos reservas activas para el mismo ejemplar.	Hace explícita la regla antes de elegir tecnología.
Tipo técnico	La interfaz de reserva es software de aplicación.	Ayuda directamente a una persona a completar una tarea.
Dominio	Educativo, pues organiza un recurso de aprendizaje.	El dominio describe la finalidad, no el lenguaje usado.
Función principal	Registrar o rechazar una solicitud según disponibilidad.	Separa el comportamiento esperado de la forma de implementarlo.

El sistema operativo que sostiene el servidor es, sin ambigüedad, software de sistema: administra recursos y sirve de base a programas. Esa clasificación no convierte por sí sola a cada componente ni a todo gestor de datos en software de sistema. También conviene separar las herramientas: un lenguaje expresa el programa, un editor ayuda a escribirlo y el runtime proporciona condiciones de ejecución.

13.2.2 2. Decisión de diseño y arquitectura

La decisión propuesta es concentrar la regla en un módulo `Reservas`. La pantalla pide `solicitar(ejemplar, persona)`; `Reservas` consulta una interfaz de `Disponibilidad` que comprueba

y marca la copia como una sola operación. Cuando hay resultado final, `Notificaciones` comunica la confirmación o el rechazo.

Esta es una decisión de **diseño** porque distribuye responsabilidades. Adquiere alcance **arquitectónico** si esa interfaz y su regla condicionan varias pantallas, equipos o despliegues. En ambos casos el análisis la precede: identifica actores, regla y resultado sin decidir todavía tablas, servidores o lenguaje.

Responsabilidad	Interfaz o entrada	Resultado que entrega
Pantalla	datos de ejemplar y persona	muestra un resultado, sin cambiar inventario
`Reservas`	`solicitar(ejemplar, persona)`	confirmación o rechazo con motivo
`Disponibilidad`	comprobar y marcar	una sola copia marcada o no disponible
`Notificaciones`	resultado final	aviso preparado para la persona

La separación reduce el acoplamiento: cambiar el medio de aviso no debe exigir que la pantalla conozca la regla de inventario. A la vez, `Reservas` mantiene cohesión si sus tareas se relacionan con decidir una reserva, no con presentar pantallas o administrar correos. No se recomienda reintentar ciegamente una solicitud: repetir una operación no idempotente podría crear otra reserva. Ante un fallo se define qué resultado quedó registrado y cómo se consulta o resuelve antes de intentar de nuevo.

13.2.3 3. Criterio de calidad y comprobación

El criterio propuesto es **consistencia** de la regla: ante dos solicitudes controladas para el mismo ejemplar disponible, se observa una confirmación, un rechazo y una sola reserva activa en el registro. Esto comprueba comportamiento funcional y consistencia de datos; no demuestra que el servicio esté disponible. La **disponibilidad** pregunta, en cambio, si el servicio puede atender solicitudes cuando falla un componente o existe una interrupción. Son cualidades relacionadas, pero no intercambiables.

> **i** Ejemplo parcial trabajado
>
> **Registro de decisión breve.** Contexto: queda un ejemplar y pueden coincidir dos solicitudes. Decisión: `Reservas` usa una operación de disponibilidad que comprueba y marca de forma atómica. Alternativa descartada por ahora: dejar que cada pantalla actualice disponibilidad; duplica una regla crítica y aumenta el riesgo de cambios inconsistentes. Consecuencia: las pantallas dependen de una interfaz estable, pero el equipo debe definir el resultado de una solicitud interrumpida.
>
> El ejemplo no decide si el sistema será monolito o microservicios. Un monolito por capas puede respetar estas fronteras; separar servicios solo sería razonable con una necesidad y costo de operación justificados.

13.2.4 4. Mito que el caso ayuda a evitar

Decir «si la demostración reserva un libro, está terminado» omite autorizaciones, registros, casos de concurrencia y condiciones de operación. La demostración aporta una señal, pero el criterio observable anterior agrega evidencia sobre una regla importante. No convierte la práctica en una garantía de seguridad, disponibilidad ni calidad total.

13.3 Ejercicio análogo: préstamo de equipos

Ahora una sala de innovación presta una cámara a estudiantes. Hay una cámara libre y dos solicitudes pueden coincidir. Propón una solución para estudiar; no implementes ni supongas que existe una plataforma institucional.

Pista opcional

Empieza por la regla que no debe romperse. Después nombra una responsabilidad que la proteja y una observación que podría refutar tu propuesta.

13.3.1 Entregables y criterios observables

Entrega en tus apuntes los cinco elementos siguientes:

1. La necesidad en una oración y la clasificación por tipo técnico, dominio y función.
2. Una distinción entre lo que descubres en análisis y lo que decides en diseño o arquitectura.
3. Una tabla pequeña con al menos dos responsabilidades, una interfaz y sus resultados.
4. Un registro de decisión con contexto, alternativa considerada, consecuencia y un estilo compatible con su justificación; no existe un estilo universalmente mejor.
5. Un criterio de calidad y una comprobación observable que distinga consistencia de disponibilidad cuando corresponda.

Puedes elegir capas, cliente-servidor o un monolito con fronteras claras si explicas qué problema atiende. Un esquema cumple cuando permite reconocer quién protege la regla, qué recibe cada parte y qué observación esperas. No cumple solo por usar nombres técnicos ni por dibujar más cajas.

13.3.2 Lista de revisión sin puntaje

- ☐ ¿Puedo señalar una regla del dominio sin nombrar una tecnología?
- ☐ ¿Distinguí la finalidad del producto de las herramientas para construirlo?
- ☐ ¿Mi interfaz evita que la pantalla cambie directamente el estado crítico?
- ☐ ¿Mi comprobación describe una entrada o condición y un resultado observable?
- ☐ ¿Expliqué una consecuencia o límite de mi decisión?
- ☐ ¿Sé qué concepto debo releer si no pude justificar una frontera?

Vuelve a necesidad, módulos y cualidades para revisar análisis, interfaces, vistas y estilos; vuelve a tipos, dominios y mitos para revisar clasificaciones y afirmaciones absolutas.

13.4 Autoevaluación formativa

Responde primero con tus palabras. Las preguntas son originales para el estudio de esta semana: no se copian de Educaplay ni representan calificación oficial.

13.4.1 1. De necesidad a decisión

Una coordinación pide «que las reservas sean claras». ¿Qué preguntas de análisis harías antes de decidir una interfaz, y cuándo explicarías que la decisión tiene alcance arquitectónico?

Retroalimentación razonada — 1

Primero aclararía actores, qué significa «clara», qué regla protege una reserva y qué resultado debe ver cada persona. Eso evita convertir una frase ambigua en una tabla o API prematura. Propondría una interfaz después de identificar la regla y los límites. La llamaría arquitectónica si condiciona a varias partes, contratos o cambios futuros; si solo organiza un detalle local, sigue siendo diseño con menor alcance.

13.4.2 2. Fronteras, acoplamiento y cohesión

Una pantalla valida disponibilidad, registra reservas y compone correos. Propón una frontera de módulos y explica qué cambio quedaría localizado.

i Retroalimentación razonada — 2

Una respuesta posible separa pantalla, `Reservas` y `Notificaciones`. `Reservas` concentra la regla y expone una operación; la pantalla presenta el resultado y Notificaciones compone el aviso. Cambiar el proveedor o formato del correo afecta principalmente a Notificaciones, por menor acoplamiento. La frontera es útil solo si las responsabilidades realmente cambian por razones distintas; dividir cada función sin necesidad puede reducir cohesión.

13.4.3 3. Vista y notación

Para explicar que una persona solicita una cámara y que el sistema la confirma o rechaza, ¿elegirías caso de uso, clases o secuencia? Indica también una vista que no resuelve por sí sola ese diagrama.

i Retroalimentación razonada — 3

Un caso de uso es una elección directa para mostrar actor y meta; una secuencia sirve si importa mostrar los mensajes del escenario. Un diagrama de clases ayuda a discutir conceptos y relaciones, no sustituye esa interacción. La vista conceptual aclara cámara, persona y préstamo; la lógica muestra componentes que colaboran; la física ubica ejecuciones y datos. Elegir una no elimina la pregunta de las otras.

13.4.4 4. Estilo con contexto

El equipo tiene una aplicación pequeña, un despliegue conocido y pocas fronteras estables. ¿Por qué «microservicios» no es automáticamente la mejor respuesta? Da una alternativa compatible.

i Retroalimentación razonada — 4

Microservicios agregan coordinación de despliegues, fallos y datos; sin una necesidad que compense esos costos pueden oscurecer el problema. Un monolito organizado por capas, atendido en cliente-servidor, puede conservar interfaces y módulos cohesivos. La elección depende de restricciones, cambios esperados y evidencia, no de una jerarquía universal de estilos.

13.4.5 5. Dos cualidades distintas

Dos solicitudes coincidentes producen una confirmación y un rechazo, pero el servicio estuvo caído una hora por un fallo externo. ¿Qué evidencia tienes y qué evidencia falta?

i Retroalimentación razonada — 5

El escenario aporta evidencia limitada de consistencia: no se prometió dos veces la misma copia. No demuestra disponibilidad, porque el servicio no atendió durante la interrupción. Para hablar de disponibilidad se definen condiciones de fallo, tiempo o nivel de servicio y cómo se observará la atención o degradación; para rendimiento se necesitaría además una carga y una medida, que esta práctica no inventa.

13.4.6 6. Tipo, dominio y herramientas

Clasifica el sistema operativo de un servidor, el portal de préstamos y el editor que usa el equipo. Luego explica por qué Python y su runtime no responden a la misma pregunta que «educativo».

i Retroalimentación razonada — 6

El sistema operativo es software de sistema; el portal es software de aplicación y puede pertenecer al dominio educativo; el editor es software de desarrollo. Python es un lenguaje y el runtime aporta el entorno para ejecutar programas: describen medios técnicos diferentes. «Educativo» describe la finalidad del portal. Por eso no se comparan como etiquetas rivales ni se infiere la categoría de una base de datos solo por su nombre.

13.4.7 7. Mito: sumar personas tarde

Un proyecto se atrasa y alguien propone añadir cinco personas mañana. ¿Qué condición revisarías antes de aceptar o rechazar la propuesta?

i Retroalimentación razonada — 7

Revisaría el cuello de botella, el tiempo para conocer el contexto y si existen tareas independientes, bien especificadas y con incorporación preparada. Si el trabajo exige coordinación intensa, sumar personas puede aumentar comunicación y errores al inicio. No es una prohibición absoluta: una tarea aislada y preparada podría repartirse. La respuesta responsable explica esa condición en lugar de prometer que más personas siempre aceleran.

13.4.8 8. Mito: «funciona, está terminado»

Una demostración registra un préstamo correctamente. Nombra dos clases de evidencia que aún podrían faltar antes de declarar terminado el producto.

i Retroalimentación razonada — 8

Podrían faltar evidencia de autorización —quién puede prestar o administrar— y de casos de falla o concurrencia, además de documentación y condiciones de entrega. Que un camino funcione confirma ese escenario, no seguridad, mantenibilidad u operación. Qué pedir depende del contexto, pero una definición explícita de terminado evita confundir una demostración con una conclusión total.

13.4.9 9. Mito: «cada cambio es fácil»

Cambiar una regla de préstamo afecta la pantalla, una interfaz compartida y registros históricos. ¿Cómo razonarías su costo sin asumir que todo cambio es difícil?

i Retroalimentación razonada — 9

Rastrearía módulos, contratos, datos existentes y personas afectadas antes de estimar, y probaría el comportamiento que depende de la regla. Es posible que un texto aislado sea fácil de cambiar; aquí las dependencias compartidas sugieren más riesgo y coordinación. La lección no es negar la evolución, sino distinguir cambios locales de cambios que atraviesan fronteras.

13.4.10 10. Mito: «no hace falta documentación»

El equipo recuerda hoy por qué rechazó que cada pantalla actualizara inventario. ¿Qué documentarías para que esa razón sobreviva a un cambio de equipo?

i Retroalimentación razonada — 10

Registraría el contexto de solicitudes coincidentes, la decisión de centralizar la regla, la alternativa de actualizar desde cada pantalla, sus consecuencias y el criterio observable de una sola reserva activa. El código puede mostrar una llamada, pero no siempre por qué se eligió ni qué supuesto debe conservarse. Una nota breve y cercana a la decisión puede ser suficiente para este alcance; no exige documentación exhaustiva sin propósito.

13.5 Cierre

La práctica se completa al poder defender una decisión con necesidad, frontera y observación, no al acertar una lista de letras. Si una respuesta fue débil, regresa al concepto enlazado, ajusta el supuesto y escribe qué evidencia pedirías después.

14. Procesos de software: actividades, flujo y trabajo reutilizable

Meta. Al terminar podrás explicar qué organiza un proceso de software, distinguir una actividad general de una tarea concreta y representar un flujo sin asumir que todos los proyectos recorren las actividades una sola vez y en el mismo orden [16].

14.1 Objetivos de aprendizaje

- Relacionar requisitos, diseño, implementación, pruebas y mantenimiento con un resultado observable.
- Explicar cómo el flujo ayuda a organizar trabajo, avance y comunicación.
- Distinguir tareas específicas de patrones reutilizables sin convertirlos en una receta universal.

Pregunta de recuperación. Si un equipo programa antes de aclarar quién puede reservar una sala, ¿qué información faltante podría obligarlo a rehacer el trabajo?

14.2 Un proceso organiza el trabajo, no reemplaza el criterio

Un **proceso de software** ofrece una forma explícita de organizar actividades para construir y mejorar un producto. Permite conversar sobre qué se intenta conseguir, qué trabajo está pendiente, qué resultado pasa a la actividad siguiente y cómo se comprobará. No garantiza calidad por sí solo: un proceso ordenado también puede producir una solución equivocada si parte de requisitos mal comprendidos o si no incorpora evidencia.

La fuente oficial presenta cinco actividades generales. Sus nombres permiten ubicar el trabajo, pero no obligan a ejecutarlas siempre como una cadena rígida. El modelo de proceso y el contexto determinan si se recorren una vez, por incrementos o en ciclos con retroalimentación y análisis de riesgos.

Actividad	Pregunta que orienta	Resultado de trabajo posible
Análisis de requisitos	¿Qué necesidad, regla y restricción debe atenderse?	requisitos aclarados y criterios observables
Diseño	¿Cómo se distribuirán responsabilidades, datos e interfaces?	decisiones y esquema de solución
Implementación	¿Cómo se construirá el comportamiento acordado?	software integrado en un alcance definido
Pruebas	¿Qué evidencia confirma o contradice lo esperado?	resultados, defectos y decisiones de corrección
Mantenimiento	¿Qué debe corregirse, adaptarse o mejorarse después de usarlo?	cambios controlados y nueva evidencia

Estas actividades se influyen mutuamente. Una prueba puede revelar que el requisito era ambiguo; una restricción descubierta durante la implementación puede llevar a revisar el diseño; el mantenimiento puede iniciar otro ciclo de análisis. Decir que existe una secuencia lógica ayuda a coordinar, pero no demuestra que toda actividad termine para siempre antes de comenzar otra.

14.3 Flujo: hacer visible cómo avanza el trabajo

El **flujo de proceso** describe cómo se relacionan y recorren las actividades. Su utilidad es práctica: ayuda a organizar tareas, controlar el avance, coordinar la comunicación del equipo y reducir errores o retrasos evitables. Para que esa ayuda sea real, el flujo debe mostrar resultados y condiciones de paso, no solo nombres de fases.

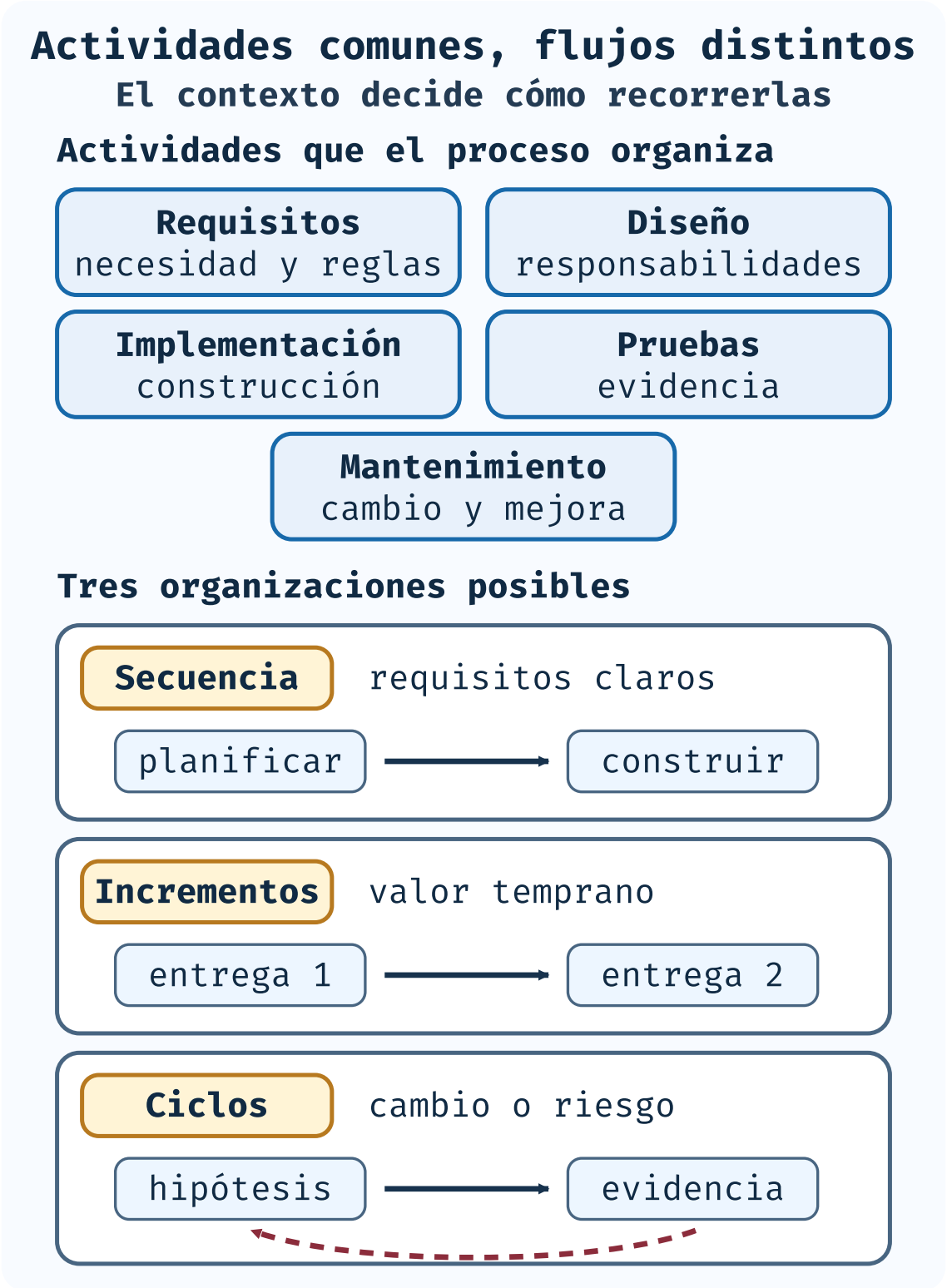


Figura 14.1: Actividades comunes del proceso y tres formas contextuales de organizarlas: una secuencia planificada, entregas por incrementos y ciclos guiados por retroalimentación o riesgo.

Figura. Las actividades pueden ser comunes, pero el recorrido depende del modelo y del contexto. Las flechas inferiores representan organizaciones alternativas; no indican que todos los proyectos deban usar simultáneamente los tres recorridos.

14.3.1 Del nombre de una fase a una condición observable

Imagina un tablero con una columna llamada «Pruebas». Mover una tarjeta allí no prueba que el sistema funcione. Una condición más útil sería: «la reserva de una sala ocupada es rechazada y no crea un registro activo». Así el equipo sabe qué evidencia buscar y puede comunicar por qué la tarea avanza, vuelve a diseño o necesita aclarar un requisito.

Un flujo también debe reconocer esperas y devoluciones. Si la biblioteca no ha definido cuánto tiempo conserva una reserva provisional, implementar un temporizador no elimina la duda. El trabajo debe regresar a la conversación de requisitos con una pregunta concreta, registrar la decisión y después actualizar diseño, código y prueba.

14.4 Tareas y patrones: distinto nivel de reutilización

Una **tarea** es una actividad específica que se ejecuta en el trabajo, como diseñar una interfaz, programar un módulo, realizar una prueba o corregir un defecto. Debe poder describirse con una entrada, una acción y un resultado. «Mejorar el sistema» es demasiado amplio; «comprobar que una reserva vencida libera la sala» permite revisar qué se hizo.

Un **patrón**, en el alcance verificado de esta semana, es una solución reutilizable para un problema común de diseño o desarrollo. Su valor está en reconocer una forma de resolver problemas recurrentes, favorecer la organización, reutilizar conocimiento y facilitar mantenimiento u optimización. No significa copiar una plantilla sin analizar consecuencias. Esta lectura no introduce un catálogo de patrones ni atribuye al contenido oficial un framework específico.

Elemento	Ejemplo en reservas de biblioteca	Comprobación
Tarea de diseño	definir la interfaz para consultar disponibilidad	la interfaz expresa entradas, resultado y errores relevantes
Tarea de implementación	programar la regla que evita dos reservas activas	el comportamiento se integra en el incremento acordado
Tarea de prueba	intentar reservar una sala ya ocupada	se obtiene rechazo y no se altera la reserva existente
Corrección de defecto	impedir que una reserva vencida siga bloqueando el horario	una prueba reproduce el fallo antes y confirma la corrección después
Solución reutilizable	separar la regla de disponibilidad de la pantalla	se reutiliza la idea solo cuando el problema y sus consecuencias coinciden

14.5 Caso guiado: primer flujo para reservar salas

Una red de bibliotecas quiere permitir que estudiantes consulten y reserven salas de estudio. Hoy cada sede lleva una hoja distinta. La primera entrega debe servir en una sede piloto; todavía no se ha decidido si las reservas podrán modificarse desde una aplicación móvil.

14.5.1 1. Aclarar el alcance inicial

El equipo formula una regla verificable: una sala no puede tener dos reservas activas en el mismo horario. También registra una incertidumbre: los cambios desde móvil no forman parte del primer alcance y deberán confirmarse con quienes usan el servicio. Eso evita diseñar una solución completa para un requisito aún no observado.

14.5.2 2. Relacionar actividades y resultados

1. **Requisitos:** identificar persona, sala, horario y regla de exclusión.
2. **Diseño:** separar consulta de disponibilidad y registro de reserva.
3. **Implementación:** construir consulta y creación para una sede piloto.

4. **Pruebas:** comprobar horario libre, horario ocupado y datos inválidos.
5. **Mantenimiento:** registrar defectos y cambios aprendidos durante el piloto.

La lista describe relaciones lógicas, no una prohibición de volver atrás. Si la prueba revela que dos solicitudes simultáneas se confirman, el equipo revisa diseño e implementación. Si las personas no entienden el estado «provisional», vuelve a requisitos y lenguaje de la interfaz.

14.5.3 3. Hacer visible el avance

Cada resultado tiene una condición de revisión. Un requisito avanza cuando incluye una regla y un ejemplo; el diseño, cuando muestra quién protege esa regla; la implementación, cuando integra un alcance ejecutable; la prueba, cuando registra el resultado; y el mantenimiento, cuando un cambio conserva la razón y la evidencia. El flujo mejora la comunicación porque el equipo puede explicar qué falta sin usar «casi terminado» como estado impreciso.

> **i** Retroalimentación del caso guiado
>
> Una respuesta razonable no necesita dibujar cinco cajas en línea. Podría organizar el piloto como un incremento y repetir análisis, diseño, implementación y pruebas para la siguiente capacidad. Lo importante es que la organización conserve la regla, muestre resultados y permita aprender. Elegir el modelo se estudia en la lectura siguiente.

14.6 Práctica breve: del trabajo difuso a tareas revisables

Para el mismo caso, el equipo escribe «hacer notificaciones» y «probar reservas». Reformula ambos elementos como tareas con entrada, acción y resultado. Después indica qué decisión debe volver a requisitos si aún no está definida.

> **Pista opcional**
>
> Separa el medio de notificación de la regla que decide cuándo se envía. En pruebas, especifica una condición y un resultado; no uses «probar todo».

> **i** Una solución posible
>
> **Tarea 1:** dada una reserva confirmada y un correo válido, preparar un aviso con sede, sala y horario; el resultado es un mensaje listo para enviar. **Tarea 2:** dada una sala con una reserva activa en un horario, solicitar el mismo intervalo; el resultado esperado es rechazo sin un segundo registro. Debe volver a requisitos la decisión sobre qué ocurre si el envío del aviso falla: la reserva podría seguir confirmada, quedar pendiente o requerir otra política. Esa regla no se deduce del nombre de la tarea.

14.7 Cierre

Un proceso aporta lenguaje para organizar trabajo y evidencia. Las actividades generales ayudan a ubicar decisiones; el flujo explica cómo avanzan, se comunican o regresan; las tareas convierten intención en trabajo revisable; y los patrones permiten reutilizar soluciones cuando el problema y sus consecuencias son comparables. La siguiente lectura usa estas bases para contrastar modelos sin declarar uno universalmente superior.

15. Modelos de proceso: comparación, práctica y autoevaluación

Propósito. Esta lectura compara cascada, incremental, evolutivo y espiral para estudiar decisiones de proceso. Incluye un caso original de reservas de biblioteca, una práctica análoga y diez preguntas con retroalimentación razonada. No reproduce ni resuelve el sistema de pedidos de restaurante ni el Taller 1.2 mencionados en la fuente oficial [16].

15.1 Ruta de estudio

1. Compara cada modelo por las condiciones que aprovecha y los problemas que debe controlar.
2. Resuelve el caso antes de abrir la decisión trabajada.
3. Justifica un modelo con requisitos, entregas, retroalimentación y riesgos; no lo elijas por popularidad.
4. Contesta la autoevaluación con tus palabras y contrasta el razonamiento.

15.2 Cuatro modelos, cuatro énfasis

Un **modelo de proceso** organiza cómo se planifican actividades, recursos, tiempos, cambios y riesgos. Su elección depende del tamaño y complejidad del proyecto, el presupuesto, el tiempo disponible y la estabilidad de los requisitos. Ninguna etiqueta elimina la necesidad de diseñar, implementar, probar y mantener con evidencia.

Modelo	Organización central	Es útil considerar cuando	Límite que debe controlarse
Cascada	fases secuenciales y documentadas	los requisitos son claros y estables, y el trabajo puede planificarse con pocas revisiones	los cambios son costosos y algunos errores pueden aparecer tarde
Incremental	entregas parciales con funciones integradas progresivamente	conviene entregar valor temprano y ampliar el sistema por partes	exige planificar fronteras e integración; el control puede volverse complejo
Evolutivo	versiones que cambian con retroalimentación frecuente	las necesidades deben descubrirse o ajustarse junto con las personas usuarias	tiempo, costo y alcance pueden ser difíciles de estimar si no se controla el aprendizaje
Espiral	ciclos de planificación, construcción y evaluación de riesgos	el proyecto es grande o complejo y existen riesgos que justifican análisis repetido	su gestión es costosa, compleja y requiere experiencia técnica

La tabla no es una puntuación. Requisitos estables favorecen una secuencia planificada, pero no vuelven infalible a cascada. Una entrega temprana favorece incrementos, pero cada incremento debe integrarse y probarse. La retroalimentación favorece evolución, pero necesita límites para no convertirse en cambios sin dirección. Un riesgo alto puede justificar ciclos de espiral, pero no toda aplicación pequeña necesita esa carga de gestión.

15.3 Cascada: previsibilidad cuando el cambio es limitado

En cascada, una fase se completa antes de iniciar formalmente la siguiente. Es fácil de explicar y favorece una organización documentada. Su supuesto más exigente es que las decisiones tempranas serán suficientemente estables. Si la biblioteca ya tiene reglas legales fijas, formatos aprobados y una única migración definida, una secuencia planificada puede ser defendible. Si las personas aún no saben cómo usarán la reserva, la validación tardía aumenta el costo de corregir el enfoque.

La desventaja no significa que el modelo «nunca pruebe» o que el equipo deba ignorar un error temprano. Significa que el flujo principal separa fases y ofrece menos flexibilidad ante cambios; por eso la calidad de requisitos y revisiones iniciales es crítica.

15.4 Incremental: entregar capacidades completas por partes

El modelo incremental divide el producto en **incrementos funcionales**. Cada entrega incorpora una parte utilizable y se integra con lo anterior. Para reservas, un primer incremento podría consultar disponibilidad; el segundo, registrar y cancelar; el tercero, notificar. El orden debe responder a valor y dependencias, no a cortar capas incompletas como «primero todas las pantallas y meses después todas las reglas».

Entregar temprano permite detectar defectos y aprender, pero no garantiza flexibilidad infinita. Una mala frontera puede duplicar reglas o dificultar la integración. La planificación define qué capacidad queda completa, qué contrato conserva y cómo se probará junto con los incrementos previos.

15.5 Evolutivo: aprender con versiones y retroalimentación

El modelo evolutivo desarrolla el software de manera progresiva según necesidades de las personas usuarias. Es adecuado cuando la interacción y el aprendizaje frecuente son centrales. Una versión inicial de reservas podría revelar que «disponible» no basta: estudiantes necesitan saber capacidad, equipamiento y accesibilidad de la sala. La retroalimentación cambia el producto y puede revisar requisitos y diseño.

La flexibilidad necesita control. El equipo registra qué hipótesis valida cada versión, qué cambio se acepta y qué evidencia permite decidir. Sin esas condiciones, el alcance puede crecer de forma desorganizada y las estimaciones pierden utilidad.

15.6 Espiral: ciclos guiados por riesgo

El modelo espiral combina desarrollo iterativo y análisis de riesgos. Cada ciclo hace visible qué incertidumbre o amenaza se estudiará antes de comprometer más trabajo. En una red grande de bibliotecas, un riesgo podría ser migrar reservas históricas sin crear dobles ocupaciones. Un ciclo puede estudiar datos, construir una prueba limitada y evaluar resultados antes de ampliar la migración.

Su ventaja es atender riesgos de forma explícita en proyectos complejos. Su costo es la planificación continua, la experiencia y el esfuerzo de evaluación. Nombrar una lista de riesgos no convierte automáticamente un proyecto en espiral: el flujo debe usar esos riesgos para orientar los ciclos y las decisiones.

15.7 Caso trabajado: reservas para una red de bibliotecas

La red quiere sustituir hojas separadas por un servicio de reservas. Se conocen la regla de no duplicar horarios y la fecha del piloto. Todavía se desconoce qué filtros necesitan estudiantes y personal. Además, migrar reservas existentes podría producir conflictos.

15.7.1 Evidencia disponible

- **Estable:** una sala no puede tener reservas activas superpuestas.
- **Necesidad temprana:** una sede debe usar el servicio en seis semanas.
- **Incertidumbre:** filtros, mensajes y flujo de modificación se aclararán con el piloto.
- **Riesgo:** datos históricos inconsistentes pueden bloquear o duplicar horarios.

15.7.2 Comparación antes de decidir

Cascada daría una secuencia clara, pero exigiría cerrar pronto requisitos de uso que aún necesitan retroalimentación. **Evolutivo** permite aprender sobre esos requisitos, aunque por sí solo no expresa qué capacidad completa se entregará en seis semanas. **Espiral** es útil para la migración riesgosa, pero podría ser una gestión excesiva para toda la interfaz del piloto. **Incremental** permite ordenar capacidades y entregar una primera utilidad, siempre que el equipo incluya puntos de retroalimentación y trate el riesgo de datos de forma explícita.

15.7.3 Decisión razonada

Para el piloto se elige un flujo **incremental**:

1. consultar salas y disponibilidad en una sede;
2. crear y cancelar una reserva sin solapamientos;
3. incorporar filtros y mensajes ajustados con la retroalimentación del piloto;
4. ampliar a otra sede solo después de evaluar la migración de sus datos.

La elección no declara que incremental sea universalmente mejor. Responde a entrega temprana, capacidades separables y aprendizaje esperado. El riesgo de migración se analiza antes de cada ampliación; eso es una práctica de gestión coherente con la comparación, no una afirmación de que todo el proyecto usa simultáneamente todos los modelos.

15.7.4 Criterios observables

- El primer incremento permite consultar horarios reales de una sede.
- El segundo rechaza solapamientos y conserva una sola reserva activa.
- La retroalimentación del piloto se registra como hipótesis, evidencia y cambio.
- La ampliación se detiene si la prueba de migración produce conflictos sin resolver.

> **i** Por qué la justificación es más importante que la etiqueta
>
> Otra respuesta podría elegir un enfoque evolutivo si la prioridad principal fuera aprender la interacción antes de fijar entregas, o espiral si la migración implicara un riesgo crítico y costoso en toda la red. Sería válida si relaciona la elección con hechos del caso, explica límites y propone evidencia. No basta con afirmar «es más moderno» o «es más seguro».

15.8 Práctica análoga: agenda de espacios comunitarios

Un municipio desea una agenda para reservar canchas y salones comunitarios. Las reglas de horario y responsables están aprobadas, pero no se conoce qué vista será más comprensible para la ciudadanía. Se necesita publicar una primera capacidad en un mes. Hay riesgo de importar duplicados desde hojas de cálculo.

15.8.1 Entregable de estudio

Escribe una página breve con:

1. hechos estables, incertidumbres y riesgos;
2. comparación de al menos dos modelos;
3. modelo elegido y razón contextual;
4. dos entregas o ciclos con resultados observables;
5. una condición que haría revisar la elección.

No implementes un sistema ni inventes métricas. La práctica evalúa razonamiento: una respuesta alternativa es válida si conserva las reglas y explica consecuencias.

Pista opcional

Separa la necesidad de publicar temprano, que favorece pensar en incrementos, de la incertidumbre de interfaz, que requiere retroalimentación. Después pregunta si el riesgo de datos justifica un ciclo específico antes de ampliar el servicio.

Ejemplo parcial de respuesta

Un primer incremento podría mostrar disponibilidad y reglas aprobadas; un segundo, permitir reservas y cancelaciones. Después de probar la vista con personas usuarias, el equipo ajustaría lenguaje y filtros. Antes de importar otra hoja, ejecutaría una prueba de duplicados y detendría la ampliación si aparecen conflictos. Esta respuesta elige incremental por entrega temprana, incorpora retroalimentación y trata el riesgo sin afirmar que una sola etiqueta resuelve todo.

15.9 Autoevaluación formativa

Responde primero sin abrir la retroalimentación. Estas diez preguntas son originales del cuaderno: no son respuestas de una evaluación, del crucigrama, del Taller 1.2 ni del caso oficial de restaurante.

15.9.1 1. Actividad y tarea

¿Cuál es la diferencia entre «pruebas» como actividad general y «comprobar que una sala ocupada no acepta otra reserva» como tarea?

Retroalimentación razonada — 1

Pruebas nombra un tipo amplio de trabajo orientado a obtener evidencia. La segunda frase define una tarea concreta con condición y resultado observable. Varias tareas pueden pertenecer a la actividad de pruebas; describirlas permite revisar avance y resultado sin suponer que una columna llamada «Pruebas» garantiza calidad.

15.9.2 2. Flujo que vuelve atrás

Una prueba revela que nadie definió cuánto dura una reserva provisional. ¿Por qué volver a requisitos no significa necesariamente que el proceso haya fracasado?

Retroalimentación razonada — 2

La prueba hizo visible una decisión faltante. Volver a requisitos permite aclarar la regla antes de consolidar una implementación arbitraria. El fracaso sería ocultar la incertidumbre o repetir trabajo sin aprender. Un flujo útil comunica devoluciones y actualiza diseño, código y pruebas con la decisión acordada.

15.9.3 3. Patrón sin receta

El equipo quiere reutilizar una solución que separa la disponibilidad de la pantalla. ¿Qué debe comprobar antes de llamarla una buena reutilización?

Retroalimentación razonada — 3

Debe comprobar que existe un problema comparable: varias interfaces necesitan una regla consistente y conviene protegerla detrás de un contrato. También debe revisar consecuencias, como coordinación y manejo de errores. Reutilizar el nombre o la forma sin compartir el problema puede añadir complejidad y no ofrece el beneficio esperado.

15.9.4 4. Cascada y estabilidad

¿Qué dato del proyecto apoyaría elegir cascada y qué dato haría esa elección más arriesgada?

i Retroalimentación razonada — 4

Apoya cascada que los requisitos, formatos y criterios estén claros, aprobados y sean poco propensos al cambio. La vuelve más arriesgada que el uso real todavía deba descubrirse o que decisiones tempranas dependan de retroalimentación tardía. La secuencia es una ventaja solo si sus supuestos coinciden con el contexto.

15.9.5 5. Incremento funcional

¿Por qué «crear todas las pantallas» no es necesariamente un buen primer incremento?

i Retroalimentación razonada — 5

Un incremento debe entregar una capacidad funcional integrada, no solo una capa sin reglas ni datos. Una consulta de disponibilidad completa puede incluir interfaz, regla, integración y prueba en un alcance pequeño. Cortar por capas puede aplazar el valor y descubrir tarde problemas de integración.

15.9.6 6. Evolución con control

¿Qué registro mínimo evitaría que un proceso evolutivo se convierta en una lista ilimitada de cambios?

i Retroalimentación razonada — 6

Conviene registrar la hipótesis o necesidad, la versión donde se observará, la evidencia recibida y la decisión de aceptar, rechazar o posponer el cambio. También se mantiene un límite de alcance o tiempo. Así la retroalimentación orienta aprendizaje en lugar de reemplazar toda planificación.

15.9.7 7. Riesgo en espiral

Un equipo enumera riesgos al inicio, pero después ejecuta ciclos idénticos sin volver a evaluarlos. ¿Qué elemento del modelo espiral está ausente?

i Retroalimentación razonada — 7

Falta usar el análisis de riesgos para orientar cada ciclo y decidir el siguiente compromiso. Una lista decorativa no reduce incertidumbre. El equipo debería elegir qué riesgo estudiar, obtener evidencia y adaptar planificación o solución según el resultado.

15.9.8 8. Dos proyectos, modelos distintos

Un sistema tiene requisitos legales estables; otro prueba una experiencia nueva con usuarios. ¿Por qué sería débil recomendar el mismo modelo solo porque ambos son web?

i Retroalimentación razonada — 8

La tecnología de interfaz no determina estabilidad, retroalimentación, riesgo ni forma de entrega. El primer contexto puede favorecer una secuencia más planificada; el segundo necesita aprendizaje frecuente. Se comparan condiciones del proyecto, no la etiqueta «web» ni una moda metodológica.

15.9.9 9. Elegir con cuatro evidencias

Para una reserva de biblioteca, nombra una evidencia sobre requisitos, otra sobre entrega, otra sobre retroalimentación y otra sobre riesgo que usarías al elegir modelo.

> **i** Retroalimentación razonada — 9
>
> Podrían ser: la regla de no solapamiento está aprobada; una sede necesita consulta en seis semanas; los filtros se definirán con el piloto; y la migración puede duplicar horarios. En conjunto favorecen entregas incrementales, puntos de aprendizaje y una evaluación explícita del riesgo antes de ampliar. Otras evidencias pueden justificar otra decisión si se explica su relación.

15.9.10 10. Revisar una elección

El equipo eligió incremental, pero descubre que ninguna capacidad parcial puede operar legalmente y todos los requisitos están fijados. ¿Qué debería hacer?

> **i** Retroalimentación razonada — 10
>
> Debe revisar la elección en vez de proteger la etiqueta. Si no existe valor ni uso legal por partes y los requisitos son estables, una secuencia planificada podría ser más coherente. También puede redefinir qué significa un incremento interno, pero no debe fingir entregas utilizables. Un modelo es una decisión contextual que cambia cuando cambian sus supuestos.

15.10 Alcance de la fuente y cierre

Esta Semana 2 desarrolla únicamente el contenido del documento HTML oficial verificado: actividades generales, flujo, tareas y patrones, y los modelos cascada, incremental, evolutivo y espiral. El plan de clase y otros archivos PDF quedaron disponibles solo como metadatos y **no fueron leídos**; por tanto, este material no afirma cubrir todas las clases, fuentes ni evaluaciones de la semana o del semestre.

La preparación se completa al poder justificar cómo se organiza el trabajo y por qué un modelo encaja en condiciones concretas. Si una respuesta depende de «siempre» o «el mejor», vuelve a requisitos, entregas, retroalimentación y riesgos: ahí debe estar la evidencia de la elección.

16. Gestión Empresarial y Emprendimiento

Este es el índice independiente de la asignatura dentro del libro compartido.

16.1 Cobertura verificada

La Semana 1 aparece publicada en el catálogo oficial, pero su cuerpo no fue leído ni convertido en material de estudio. Por ello, **no hay capítulos de estudio verificados y redactados** para esta asignatura en la edición actual.

16.2 Ruta de lectura

Cuando exista una fuente oficial legible y verificada, este índice enlazará sus capítulos por semana. Hasta entonces, no se infieren temas, calendario, actividades ni respuestas a partir del nombre de la asignatura.

17. Inteligencia Artificial

Este es el índice independiente de la asignatura dentro del libro compartido.

17.1 Cobertura verificada

La Semana 1 aparece publicada en el catálogo oficial, pero su cuerpo no fue leído ni convertido en material de estudio. Por ello, **no hay capítulos de estudio verificados y redactados** para esta asignatura en la edición actual.

17.2 Ruta de lectura

Cuando exista una fuente oficial legible y verificada, este índice enlazará sus capítulos por semana. Hasta entonces, no se infieren teoría, prácticas, calendario ni respuestas a partir del nombre de la asignatura.

18. Inteligencia de Negocios

Este es el índice independiente de la asignatura dentro del libro compartido.

18.1 Cobertura verificada

El módulo de contenidos observado no tenía material publicado. Por ello, **no hay capítulos de estudio verificados y redactados** para esta asignatura en la edición actual.

18.2 Ruta de lectura

Cuando exista contenido oficial legible y verificado, este índice enlazará sus capítulos por semana. No se inventan temas, actividades, calendario ni respuestas para completar una sección vacía.

19. Proyectos Informáticos

Este es el índice independiente de la asignatura dentro del libro compartido.

19.1 Cobertura verificada

Solo se observó metadata de objetivos; no se verificó un cuerpo docente suficiente para redactar teoría o práctica. Por ello, **no hay capítulos de estudio verificados y redactados** para esta asignatura en la edición actual.

19.2 Ruta de lectura

Cuando exista una fuente oficial legible y verificada, este índice enlazará sus capítulos por semana. La metadata disponible no autoriza a inferir causas, métodos, actividades, calendario ni respuestas.

20. Fuentes y política de uso

20.1 Política de fuentes

Los contenidos de Semana 1 de **Aplicaciones Web con PHP y C#** se organizan primero a partir de su documentación oficial [5], [6], [7], consultada el **30 de octubre de 2026**. Los enlaces pueden pedir autenticación institucional. Este libro los enlaza y parafrasea de forma original; no copia ni redistribuye material privado del aula.

La Semana 1 de **Arquitectura de Software** se organiza a partir de su fuente oficial de contenidos [15], consultada el **31 de octubre de 2026**. La explicación editorial desarrolla esos temas con redacción original. Como enriquecimiento técnico, Parnas fundamenta la ocultación de decisiones de cambio [17] y Lehman trata la evolución como un fenómeno contextual [18]; ninguna de las dos fuentes sustituye el currículo oficial.

Las fuentes técnicas complementarias de Aplicaciones Web se usan para verificar sintaxis, plataformas y herramientas, no para ampliar el currículo más allá de Semana 1. Cuando haya una diferencia entre una explicación editorial y una fuente primaria, prevalece la fuente primaria. Los ejemplos de los repositorios de práctica se identifican como evidencia de trabajo existente [8], [19], no como una afirmación de que todo el código siga MVC ni como requisito adicional.

Cada entrada conserva su propia fecha de consulta cuando corresponde. La bibliografía distingue las fuentes oficiales de cada asignatura del enriquecimiento técnico suplementario.

20.2 Bibliografía

Bibliografía

[1] The PHP Group, «PHP Manual». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://www.php.net/manual/en/

[2] Microsoft, «C sharp language reference». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/

[3] The PHP Group, «PHP 8.5». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://www.php.net/releases/8.5/en.php

[4] Microsoft, «ASP.NET Core MVC overview». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://learn.microsoft.com/en-us/aspnet/core/mvc/overview?view=aspnetcore-10.0

[5] UNIANDES (Ecuador), «Clase 1: aplicaciones web y preparación del entorno». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://uniandesec.blackboard.com/ultra/courses/_3085_1/document/_334077_1?view=content&state=view

[6] UNIANDES (Ecuador), «Clase 2: PHP, C sharp y variables». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://uniandesec.blackboard.com/ultra/courses/_3085_1/document/_334079_1

[7] UNIANDES (Ecuador), «Evaluación de Semana 1: página personal y calculadora PHP». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://uniandesec.blackboard.com/ultra/courses/_3085_1/assessment/_334154_1/overview?courseId=_3085_1

[8] statick88, «aplicaciones-web-php-csharp: repositorio de práctica». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://github.com/statick88/aplicaciones-web-php-csharp/tree/406f4c02b3803f4ec444559239a8d93c724773c8

[9] Dev Container Specification, «Development containers». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://containers.dev/

[10] Microsoft, «Developing inside a Container». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://code.visualstudio.com/docs/devcontainers/containers

[11] Docker, «Docker Engine installation». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://docs.docker.com/engine/install/

[12] Microsoft, «Quickstart: Run SQL Server Linux container images with Docker». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://learn.microsoft.com/sql/linux/install-upgrade/quickstart-install-docker

[13] L. Llerena, «Semana 2, Clase 1: Tipos de datos simples y variables de variables».

[14] L. Llerena, «Semana 2, Clase 2: Envío de variables por formulario, métodos GET y POST».

[15] UNIANDES (Ecuador), «Semana 1 — Clase 1: Contenidos». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://uniandesec.blackboard.com/ultra/courses/_3086_1/document/_335492_1?view=content&state=view

[16] UNIANDES (Ecuador), «Semana 2 — Clase 1: Contenido sobre procesos y modelos de software». Accedido: 1 de noviembre de 2026. [En línea]. Disponible en: https://uniandesec.blackboard.com/ultra/courses/_3086_1/document/_335498_1?view=content&state=view

[17] D. L. Parnas, «On the Criteria To Be Used in Decomposing Systems into Modules», *Communications of the ACM*, vol. 15, n.° 12, pp. 1053-1058, 1972, doi: 10.1145/361598.361623.

[18] M. M. Lehman, «Laws of Software Evolution Revisited», en *Software Process Technology*, vol. 1149, C. Montangero, Ed., en Lecture Notes in Computer Science, vol. 1149., Springer, 1996, pp. 108-124. doi: 10.1007/BFb0017737.

[19] statick88, «aplicaciones-web-semana1: entrega de práctica». Accedido: 30 de octubre de 2026. [En línea]. Disponible en: https://github.com/statick88/aplicaciones-web-semana1/tree/ff3d1d757b0799bdad2b2a8f4813f3023f3f49ca